Zarządzanie w systemach i sieciach komputerowych

Dr inż. Robert Wójcik

Wykład 6. Problem martwego punktu i zagłodzeń w systemach procesów współbieżnych.

6.1. Systemy procesów współbieżnych.

6.2. Problemy rozstrzygania konfliktów zasobowych – martwy punkt (impas) i zagłodzenie.

6.3. Klasy problemów rozstrzygania konfliktów zasobowych

6.4. Metody i algorytmy rozwiązywania problemu martwego punktu

- 6.4.1. Metody wykrywania i likwidacji impasów
- 6.4.2. Metody zapobiegania impasom
- 6.4.3. Metody unikania impasów
- 6.4.4. Inne rodzaje algorytmów

6.5. Przegląd wybranych rozwiązań

- 6.5.1. Algorytmy scentralizowane
- 6.5.2. Algorytmy rozproszone

6.1. Systemy procesów współbieżnych

Proces - zwykle uporządkowany w czasie ciąg zmian (zdarzeń) i stanów zachodzących po sobie, zmian właściwości lub wartości zmiennych opisujących dany obiekt (system, zasób) fizyczny. Każdy kolejny stan/zmiana systemu spowodowana jest bądź to przez poprzedni stan/zmianę bądź też przez oddziaływanie zewnętrzne.

[1] Banaszak Z., Majdzik P., Wójcik R., Procesy współbieżne. Modele efektywności funkcjonowania, Wyd. Politechniki Koszalińskiej, Koszalin 2008.

Procesy ciągłe i dyskretne

Procesy ciągłe to takie gdzie na dowolnym skończonym odcinku czasu wyróżnić można nieskończoną liczbę zmian, przy czym różnice między nimi mogą być nieskończenie małe.

Procesy dyskretne, z kolei to takie, w których wyróżnić można tylko skończoną liczbę zdarzeń/stanów, przy czym różnice między nimi mają skończoną wartość.

W informatyce w wielu zastosowaniach mamy do czynienia z dyskretnymi procesami sekwencyjnymi, w których relacją determinującą kolejność występowania stanów jest relacja porządku liniowego.

- Przykładem procesu sekwencyjnego jest wykonujący się program komputerowy, którego realizacja, jak sama nazwa wskazuje, przebiega sekwencyjnie, tzn. w każdej chwili wykonywany jest tylko jeden rozkaz kodu programu.

- Innym przykładem procesu sekwencyjnego może być przesyłanie pakietów w sieci komputerowej wzdłuż zadanej trasy.

- Kolejnym przykładem może być sekwencyjny proces produkcyjny, w którym kolejne operacje obróbkowe, wykonywane na odpowiednich maszynach, prowadzą do powstania gotowego elementu.

- Procesy sekwencyjne mogą występować również w systemach transportowych, np. systemy, w których realizowane są powtarzalne operacje transportowe, wykorzystujące wspólne trasy lub środki transportu, zgodnie z ustalonym rozkładem jazdy (metro, kolej, komunikacja miejska) i zasadami dostępu do tras przejazdu (np. zasady ruchu ulicznego).

W wielu zastosowaniach współbieżne procesy sekwencyjne:

- mają charakter cyklicznych procesów powtarzalnych,

- współdzielą zasoby w trybie wzajemnego wykluczania,

- podczas wykonywania operacji nie mogą być wywłaszczone z zasobów,

- przetrzymują zasoby, aż do momentu uzyskania rezerwacji zasobów niezbędnych do wykonania kolejnej operacji (np. sekwencyjne transakcje bazodanowe).

System procesów

System procesów rozumiany jest jako zbiór procesów

$P = \{ p_i \mid i=1,...,n\}$

realizujących operacje w oparciu o

zbiór wspólnie wykorzystywanych zasobów $R = \{ r_k \mid k=1,...,m\}$.

Przyjmuje się, że system procesów dodatkowo charakteryzują:

$C(r_k)$ - liczba dostępnych jednostek zasobu r_k , (pojemność zasobu),

$Z_I = <O_{I1}, O_{I2}, ... , O_{IJ}, ... , O_{IK}>$ - sekwencja operacji realizowanych przez proces p_i (i-ty program).

Każda operacja o_{ij} i-tego procesu realizowana jest w oparciu o zasoby należące do zbioru $r(o_{ij})$. Zakłada się, że do wykonania operacji o_{ij} niezbędnych jest $w(r_k,o_{ij})$ jednostek zasobu r_k ze zbioru $r(o_{ij})$.

W ogólnym przypadku wykonanie poszczególnych operacji może być oczywiście warunkowane dostępnością odpowiedniego podzbioru zasobów. Ponadto, przyjmuje się, że procesy wykonywane są w trybie współbieżnym, tzn. dostęp do zasobów odbywa się zgodnie z zasadą wzajemnego wykluczania.

Przykładowy system procesów i ich programów ma postać: P = { p1 , p2 }, gdzie:

p1 : < (2r1, 1r2), (2r3), (2r3, 3r4) > - program procesu p1 (operacje o1, o2, o3),

p2 : < (1r2), (1r3, 3r5) > - program procesu p2 (operacje o4, o5),

r(o1) = {r1, r2}, w(r1, o1) = 2, w(r2, o1) = 1,
r(o2) = {r3}, w(r3, o2) = 2,
r(o3) = {r3, r4}, w(r3, o3) = 2, w(r4, o3) = 3,
r(o4) = {r2}, w(r2, o4) = 1,
r(o5) = {r3, r5}, w(r3, o5) = 1, w(r5, o5) = 3,

C(r1)=C(r2)=2, C(r3)=C(r4)=C(r5)=3.

Procesy współbieżne

Zadania systemu procesów mogą być realizowane współbieżnie (np. na jednym komputerze z podziałem czasu lub na wielu komputerach – równolegle) z wykorzystaniem wspólnych zasobów.

Współbieżne wykonanie procesów oznacza, że każda kolejna operacja (akcja) jednego procesu rozpoczyna się przed zakończeniem operacji innego procesu, a ich możliwe oddziaływanie związane jest z ograniczonym dostępem procesów do współdzielonych zasobów.

Inaczej mówiąc, dwa procesy traktowane są jako współbieżne, jeżeli każdy z nich rozpoczął się przed zakończeniem drugiego, a ich wykonaniu towarzyszy konieczność wzajemnej interakcji.

Równoczesna realizacja wielu procesów (zadań) w oparciu o wspólne zasoby ma na celu zwiększenie efektywności działania systemów, w szczególności skrócenie czasu realizacji zadań.

W przypadku jednak współdzielenia zasobów w trybie wzajemnego wykluczania w systemie procesów współbieżnych może dochodzić do konfliktów zasobowych. Jeśli konflikty te będą rozwiązywane w sposób niewłaściwy, tj. niewłaściwy będzie mechanizm synchronizacji dostępu procesów do współdzielonych zasobów (mechanizm rozstrzygania konfliktów zasobowych), to mogą pojawić się zabronione warianty działania systemu procesów, prowadzące do powstawania martwego punktu (impasu, zastoju, blokady) lub głodzenia procesów.

Harmonogramom działania procesów

Zachowanie systemu procesów można opisać za pomocą harmonogramu, który określa chwile rozpoczęcia oraz zakończenia poszczególnych operacji procesów w ustalonym horyzoncie czasowym.

Harmonogramu działania systemu procesów jest prawidłowy jeśli gwarantuje, spełnienie wszystkich ograniczeń jakościowych oraz ilościowych zdefiniowanych w systemie.

Przykładowe ograniczenia w systemach procesów współdzielących zasoby:

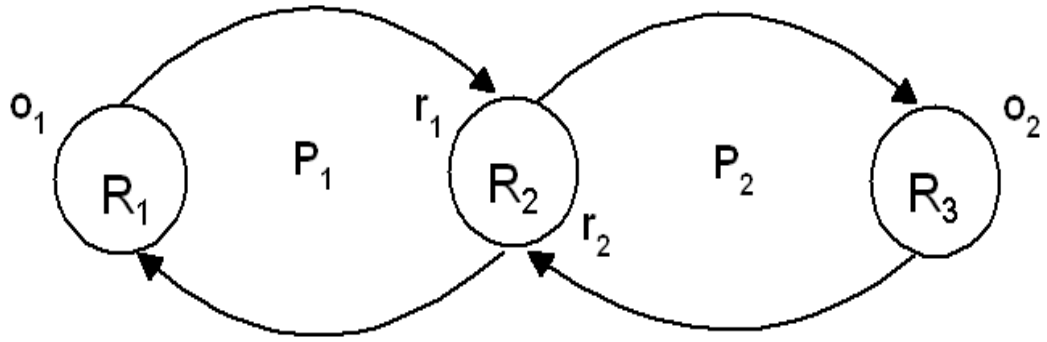
- nie występowanie blokad procesów (procesy powinny zakończyć operacje w skończonym czasie);

- nie występowanie głodzenia procesów (czas oczekiwania procesu na dostęp do zasobu powinien być ograniczony);

- spełnienie wszystkich ograniczeń kolejnościowych;

- spełnienie wszystkich ograniczeń czasowych, np. zachowanie odległości czasowych między operacjami, nie przekraczanie czasów dostępności zasobów.

Przykładowy harmonogram działania systemu procesów cyklicznych



Proces 1: $Z_1 = (R_1,R_2) = (1, 2)$;
Czasy operacji: $ZT_1 = (o_1, r_1)$; $o_1, r_1 \in Z_+$ - liczby całkowite;
$o_1 = 4$ – czas operacji wykonywanej na zasobie własnym R_1;
$r_1 = 3$ – czas operacji wykonywanej na zasobie dzielonym R_2;
$c_1 = r_1 + o_1 = 7$ – czas cyklu procesu P_1.

Proces 2: $Z_2 = (R_3,R_2) = (3, 2)$;
Czasy operacji: $ZT_2 = (o_2, r_2)$; $o_2, r_2 \in Z_+$ - liczby całkowite;
$o_2 = 3$ – czas operacji wykonywanej na zasobie własnym R_3;
$r_2 = 2$ – czas operacji wykonywanej na zasobie dzielonym R_2;
$c_2 = r_2 + o_2 = 5$ – czas cyklu procesu P_2.

Priorytet: według czasu przyjścia (np. FIFO); przy żądaniach równoczesnych decyduje mniejszy numer procesu.

Stan początkowy: $S_0 = (P_1, 0, P_2)$,
P_1 – zasób R_1 (1), P_2 – zasób R_3 (3).
Fazy początkowe (przesunięcie względem zera): $(x_1(0), x_2(0)) = (0, 0)$.

Pojawia się cykliczny przebieg ustalony o czasie cyklu T = 7, w którym proces P_2 oczekuje na dostęp do zasobu wspólnego (cykle składowe $c_1 = 7$ i $c_2 = 5$).



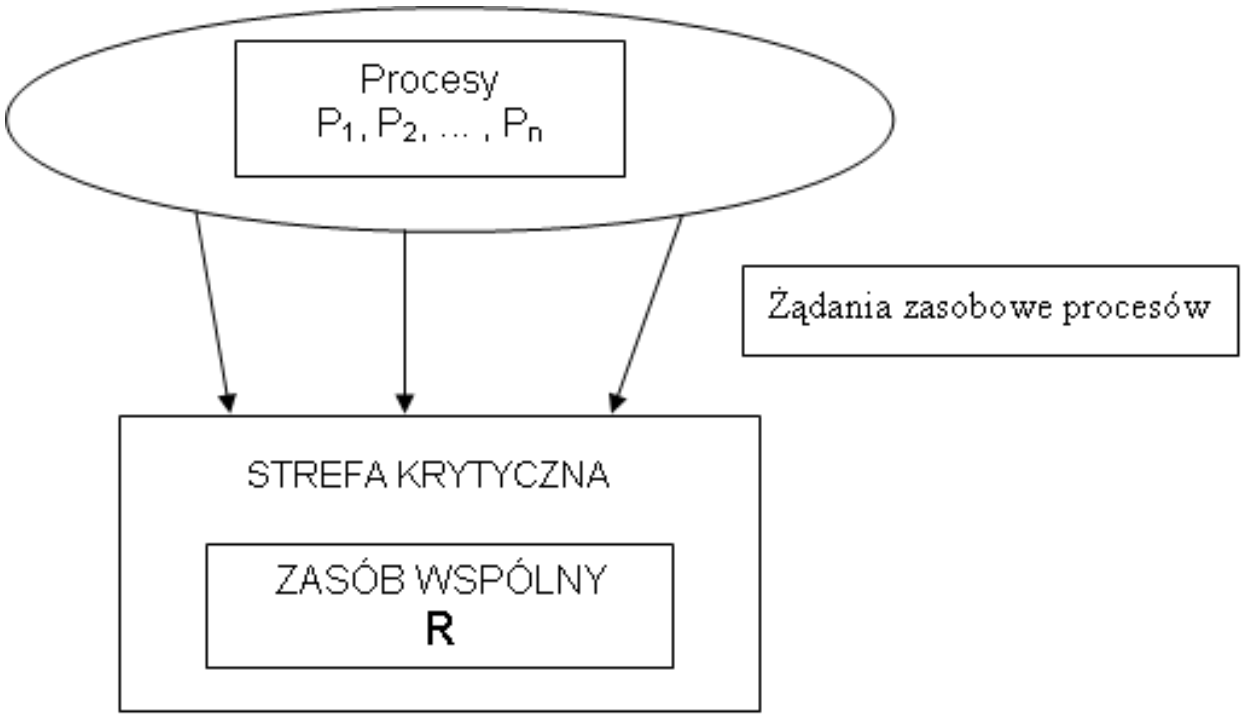
6.2. Problemy rozstrzygania konfliktów zasobowych – martwy punkt (impas) i zagłodzenie

Problem rozstrzygania konfliktów zasobowych:

- procesy mogą równocześnie zgłaszać żądania dostępu do wspólnych zasobów, tj. w systemie procesów mogą pojawiać się konfliktowe żądania zasobowe;

- problem rozstrzygania konfliktów zasobowych, sprowadza się do określenia kolejności dostępu konfliktowych procesów do zasobów, tak aby zagwarantować ***prawidłowe*** działanie systemu.

W szczególności, można przydzielać zasoby wspólne zgodnie z zasadą „wszystko albo nic", która zapewnia wzajemne wykluczanie w dostępie do tych zasobów, ale nie jest regułą gwarantującą wysoki stopień współbieżności.



Wykorzystywane w praktyce różne mechanizmy synchronizacji wyrażają się w różnych koncepcjach implementacji zasady wzajemnego wykluczania (ang. mutual exclusion), inaczej mówiąc odpowiedniego sterowania dostępem do współdzielonych zasobów.

W chwili korzystania z takiego zasobu proces wchodzi do tzw. sekcji krytycznej (ang. critical region), np. wykonując pewien szczególny fragment swojego kodu.

Zgodnie z zasadą wzajemnego wykluczania, w celu eliminacji konfliktowych żądań zasobowych, w tym samym czasie w sekcji krytycznej może działać tylko jeden proces.

W ogólnym przypadku w sekcji krytycznej (determinującej ograniczenia zasobowe systemu) może działać więcej procesów, nie więcej jednak niż umożliwia to zakładana, dopuszczalna przepustowość (pojemność, wydajność, itp.) odpowiedniego zasobu.

W analogii do ruchu ulicznego, sekcję (strefę) krytyczną przyrównać można do skrzyżowania ulic. Przyjmując, że procesami są samochody usiłujące przejechać przez skrzyżowanie (np. na wprost) należy zapewnić, że nie dojdzie do kolizji (na skrzyżowaniu nie znajdą się jednocześnie samochody jadące w prostopadłych kierunkach.

Synchronizacja ruchu na jednym skrzyżowaniu musi być również skoordynowana z synchronizacjami innych skrzyżowań. Wyznaczenie takiego bezpiecznego (niedopuszczającego do kolizji, blokad - korków i zagłodzeń) sposobu koordynacji nie jest zadaniem trywialnym.

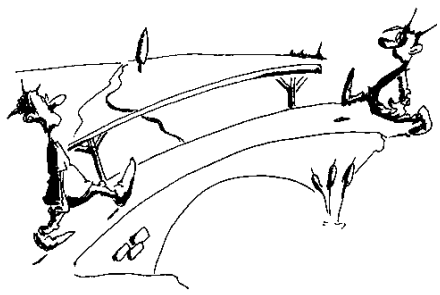
Problem impasów (zastoju)

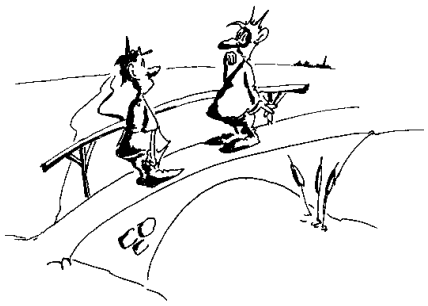
Stan martwego punktu (blokady, zastoju, impasu) występuje wówczas, gdy kilka (przynajmniej dwa) współbieżnych procesów czeka na siebie wzajemnie, oczekując na wykonanie operacji, która nigdy nie nastąpi.

Źródłem blokad w systemach ze współdzielonymi zasobami jest między innymi niewłaściwy sposób organizacji dostępu do strefy krytycznej lub konieczność spełnienia następującego wymogu: proces może zwolnić zajmowany (już wykorzystany) zasób dopiero, gdy zdoła zająć zasób następny.

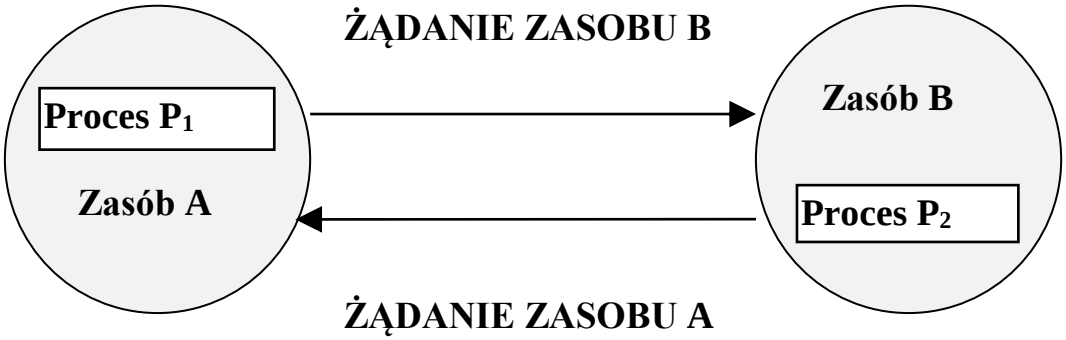
Przykład – kładka, która może pomieścić tylko jedną osobę.

Sytuacja, w której każdy przechodzień nastąpił na swój koniec kładki oznacza ich zastój – nie mogą kontynuować swoich marszrut.





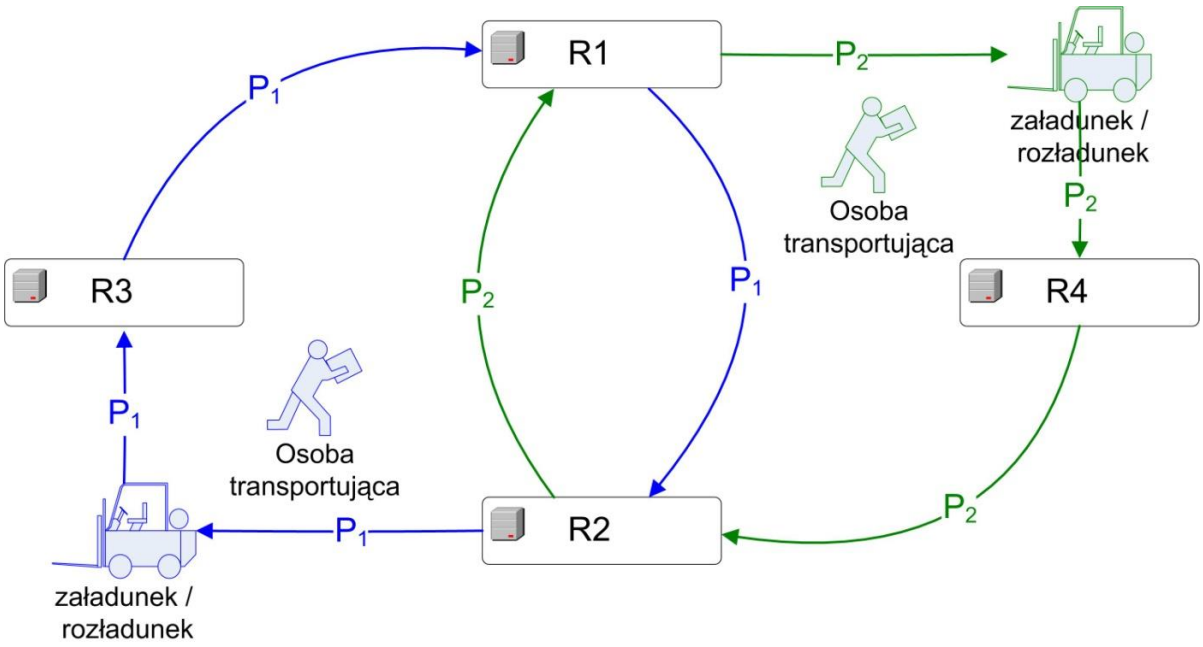
W sposób bardziej formalny stan blokady ilustruje następujący przykład. Dany jest proces P1, zajmujący zasób A, żądający dostępu do zasobu B. Zasób ten jest zajęty przez proces P2, który z kolei żąda dostępu do zasobu A, zajmowanego właśnie przez proces P1. Powstaje zamknięty łańcuch żądań zasobowych - stan blokady, który zatrzymuje poprawne funkcjonowanie systemu.



System procesów cyklicznych z cyklami żądań zasobowych

W przypadku, gdy graf opisujący strukturę systemu procesów zawiera cykle, złożone z zasobów wspólnych, może pojawić się stan blokady, w którym istnieje cykl żądań zasobowych, z których żadne nie może być zrealizowane, np. jeśli równocześnie proces P_1 zajmie zasób R_1, a proces P_2 zasób R_2.

Wystąpienie stanu blokady zależy w tym przypadku od stanu początkowego systemu (fazy startowej procesów cyklicznych).



Proces 1: zasoby - $Z_1 = (R_3,R_1,R_2) = (3, 1, 2)$; czasy operacji - $ZT_1 = (4,2,1)$;

Proces 2: zasoby - $Z_2 = (R_4,R_2,R_1) = (4, 2, 1)$; czasy operacji - $ZT_2 = (2,3,1)$.

Priorytet: przy żądaniach równoczesnych dostęp do R_1 ma P_2, do R_2 ma P_1;

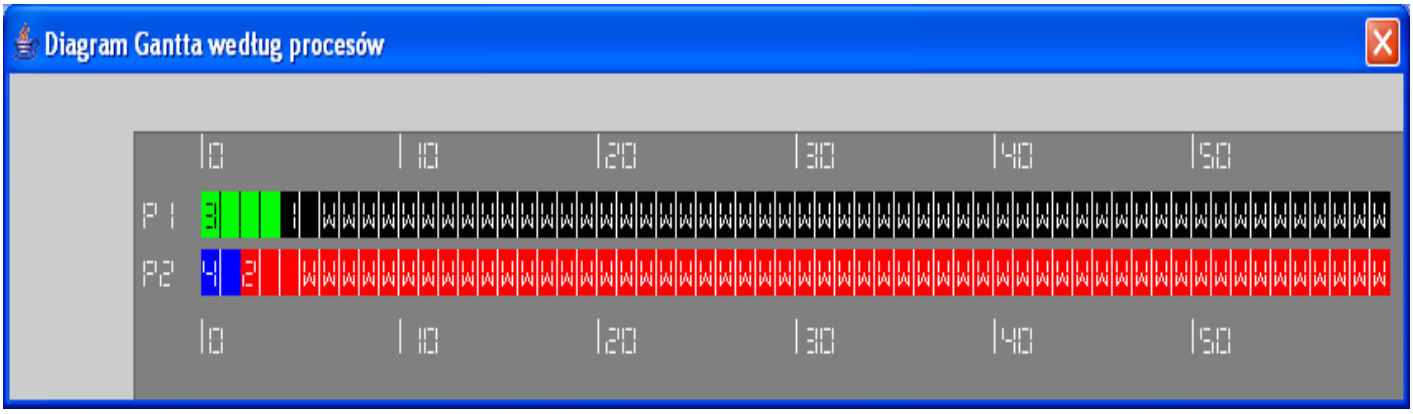
Stan początkowy: $S_0 = (0, 0, P_1, P_2)$, P_1 – zasób R_3 (3), P_2 – zasób R_4 (4).

Fazy początkowe (przesunięcie względem zera): $(x_1(0), x_2(0)) = (0, 0)$.

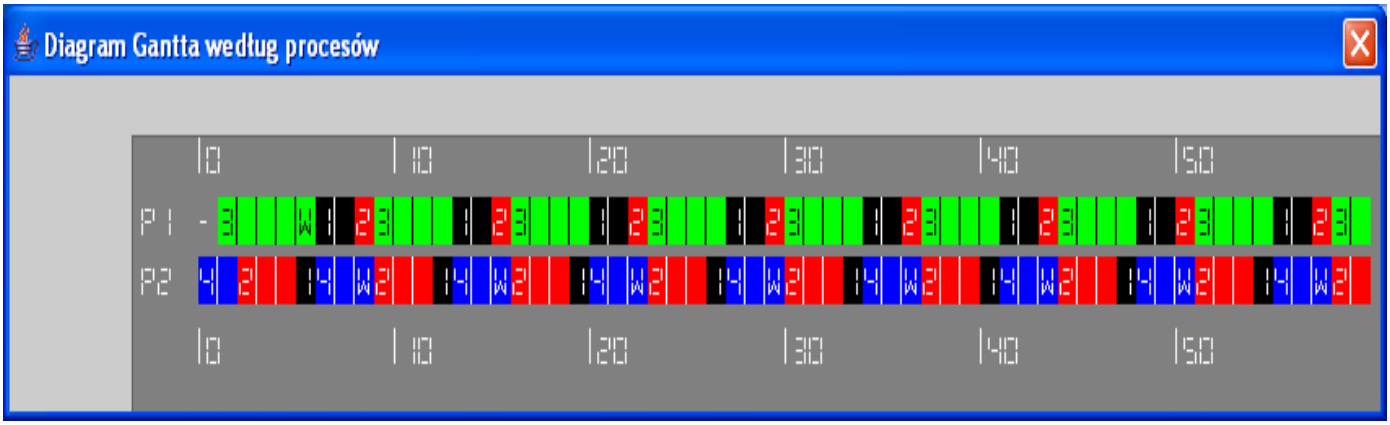
Dla podanych parametrów systemu i faz początkowych (0,0)
pojawia się stan blokady.

Jednak zmieniając fazę początkową na $(x_1(0), x_2(0)) = (1, 0)$, otrzymamy harmonogram bezblokadowy o okresie T=7.

Harmonogram blokadowy; faza początkowa: (0,0)

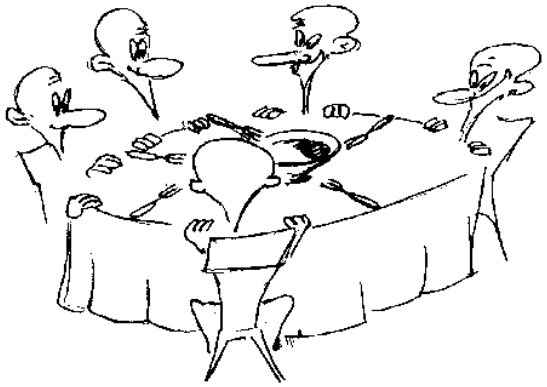


Harmonogram bezblokadowy; faza początkowa (1,0)



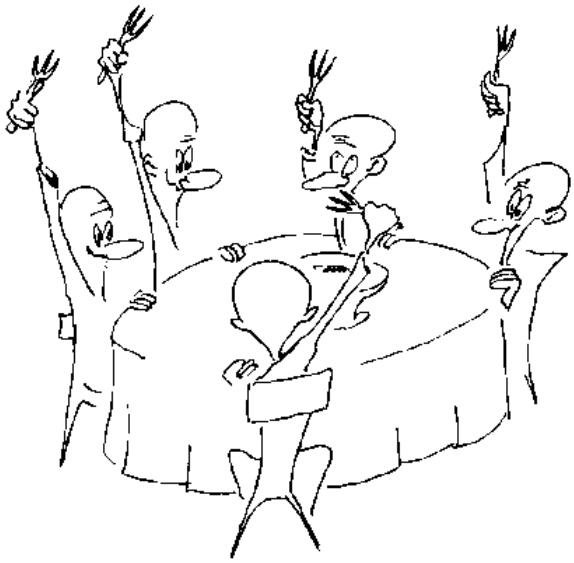
Problem pięciu filozofów

Pięciu filozofów siedzących za wspólnym stołem na przemian je i rozmyśla. Na stole do dyspozycji jest 5 talerzy i 5 widelców. Przy stole panuje zwyczaj, że jedzący filozof posługuje się dwoma widelcami, zwracając je do wspólnego użytkowania po zakończeniu posiłku. Problem polega na takiej synchronizacji poczynań filozofów, aby wykluczyć możliwość powstania impasu i zagłodzenia.



Stan blokady

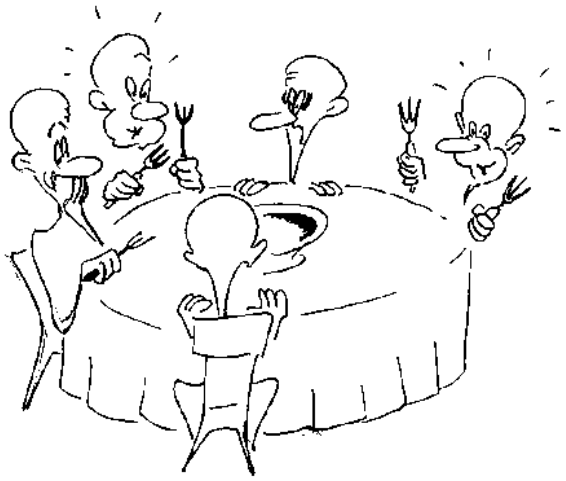
Łatwo widać, że rozwiązanie umożliwiające filozofom kompletowanie sztućców przez ich pojedyncze pobieranie może doprowadzić do blokady.



Stan zagłodzenia

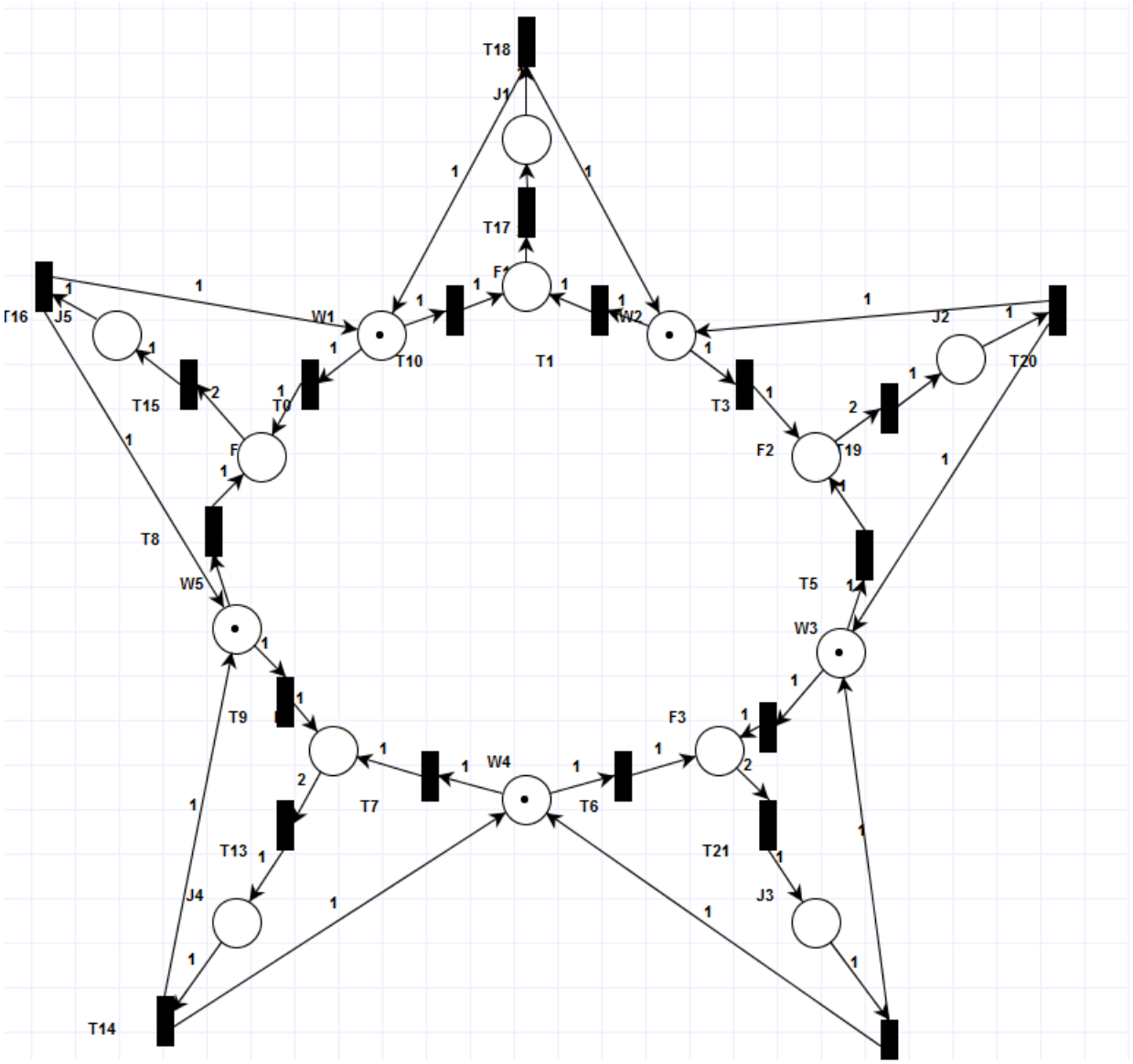
Z kolei, przyjęcie rozwiązania, umożliwiającego filozofom pobieranie jednocześnie tylko dwóch widelców prowadzić może do zagłodzenia jednego z filozofów.

Załóżmy, że jeden z filozofów czeka na widelec ze swojej prawej strony. W tym czasie jego kolega filozof z lewej strony zaczyna jeść, zabierając mu wolny dotychczas widelec. Po chwili filozof z prawej strony odkłada swój widelec, z którego nie można skorzystać, bo ciągle brak jest jeszcze widelca zajmowanego przez filozofa siedzącego z lewej strony. Po chwili filozof z prawej strony znowu zaczyna jeść, zabierając widelec z prawej strony. Filozof z lewej strony kończy jeść i odkłada widelec, niestety z prawej strony brak wolnego widelca, itd. W ten sposób filozof będzie czekał raz na widelec z lewej, raz prawej strony, ulegając (dosłownie) zagłodzeniu.



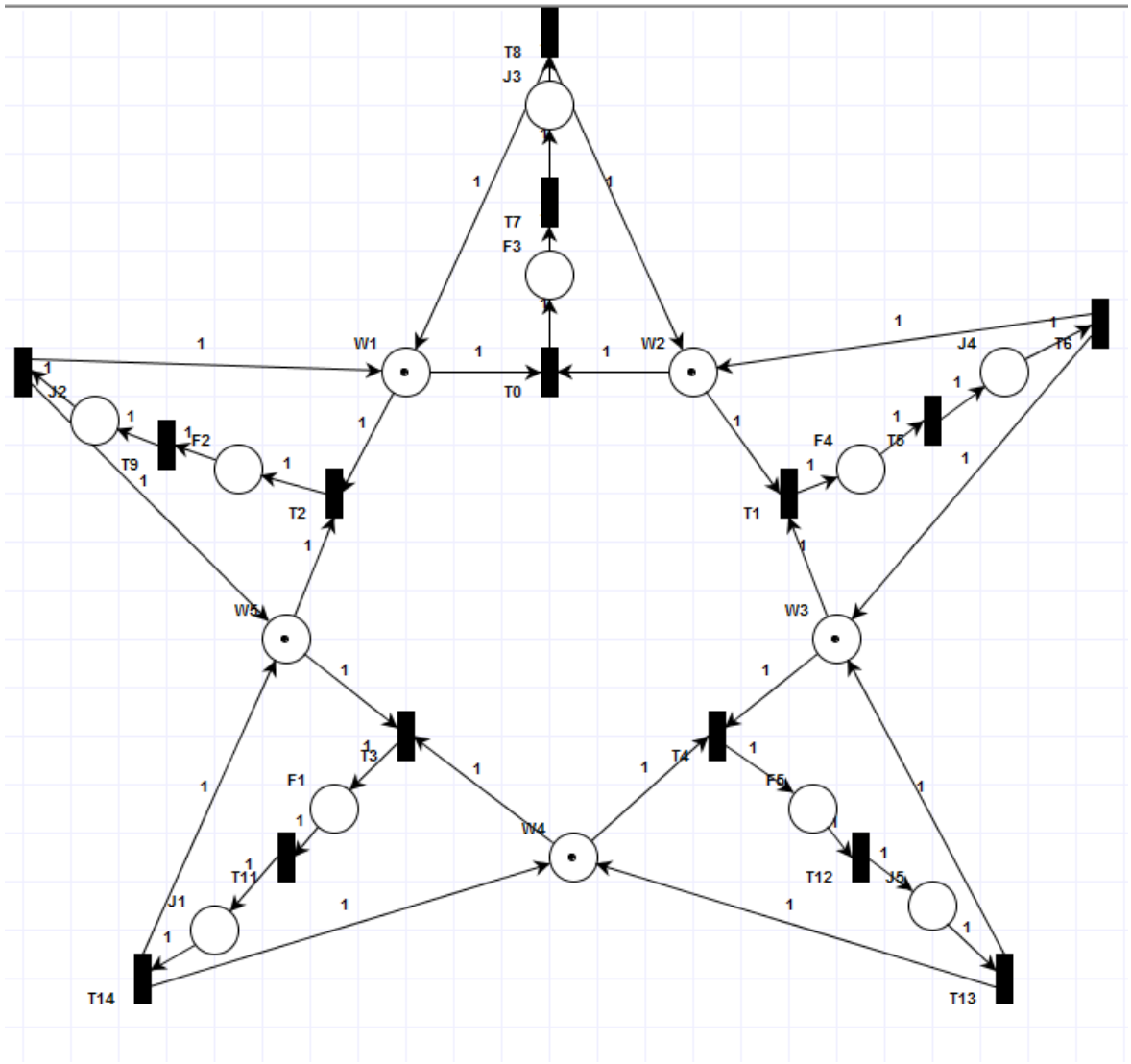
Możliwe metody (protokoły) synchronizacji procesów w systemie złożonym z pięciu filozofów (**zakładamy, że każdy filozof działa w ten sam sposób**).

- Protokół z możliwością blokady: każdy filozof może podnieść niezależnie lewy lub prawy widelec, a następnie przejść do oczekiwania na kolejny.



- Protokoły bezblokadowe, ale z możliwością zagłodzenia procesu:

  - każdy filozof zawsze musi podnieść dwa widelce jednocześnie;



  - każdy filozof podnosi najpierw lewy widelec, a potem prawy widelec (lub odwrotnie), ale zawsze maksymalnie tylko 4 filozofów może podnieść jeden lewy (prawy) widelec.

- Protokoły nieprowadzące do blokad i zagłodzeń:
  - każdy filozof musi podnieść dwa widelce jednocześnie, ale każdy z nich musi zjeść raz (w dowolnej kolejności) i potem następuje kolejna tura jedzenia
  - każdy filozof musi podnieść jednocześnie dwa widelce, ale jest priorytet cykliczny (round-robin), określający kolejność jedzenia.

Przedstawiony przykład można interpretować, jako szczególny przypadek sytuacji, w której zbiór procesów (filozofów) do swoich realizacji wymaga jednoczesnego dostępu do dwóch typów zasobów (np. pamięci i drukarki). Przy założeniu ograniczonej ilości współdzielonych zasobów dochodzi do konfliktów związanych z ich przydziałem. Skutki niewłaściwego rozstrzygania tych konfliktów np. metodą ad hoc (dowolny wybór widelców) mogą prowadzić do zagłodzenia lub blokady procesów.

System obsługi pożyczkobiorców – problem bankiera

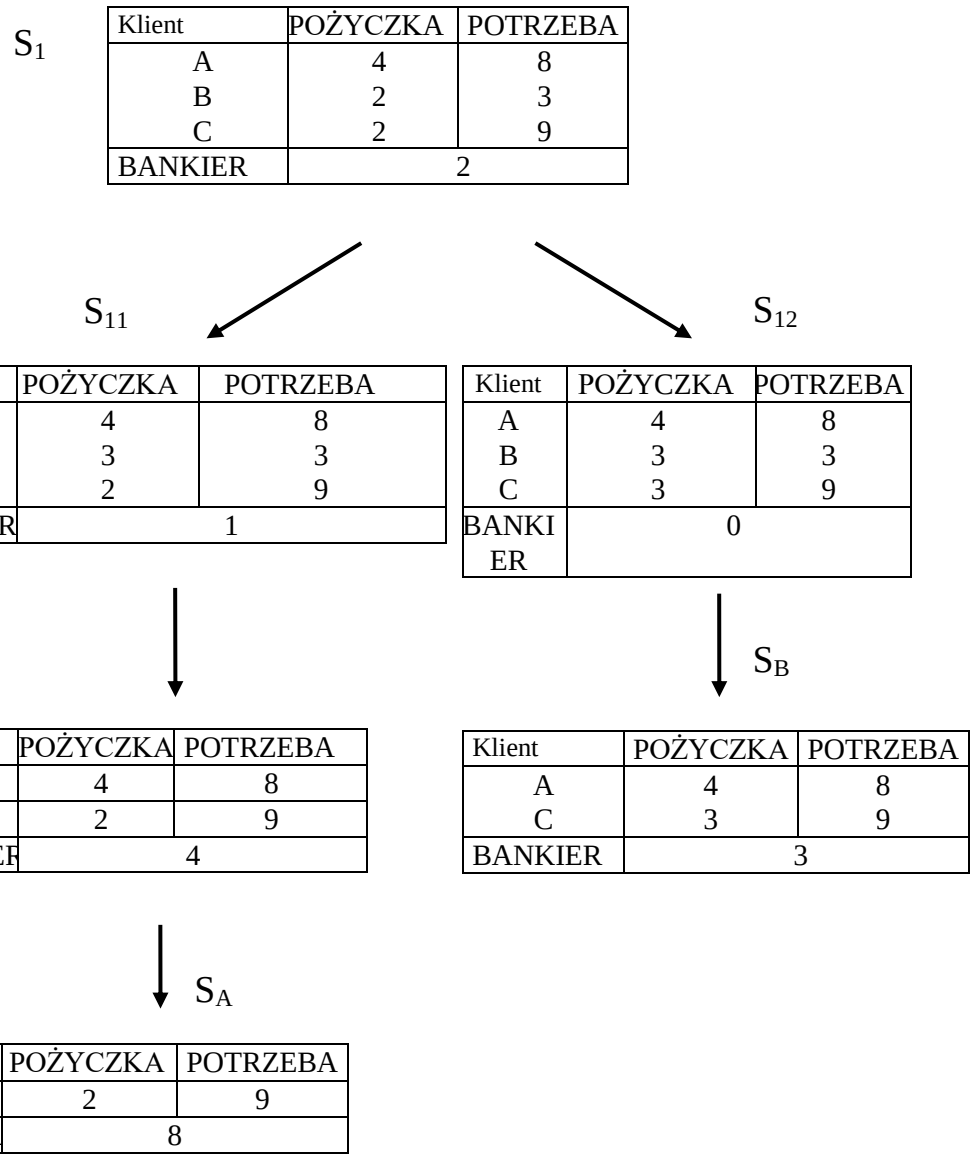
Rozważana jest sytuacja, w której BANKIER dysponujący kapitałem 10 jednostek winien obsłużyć trzech KLIENTÓW A, B i C zgłaszających swoje POTRZEBY w wysokości odpowiednio 8, 3 i 9 jednostek umownej waluty. Przyjmując, że POŻYCZKA udzielona każdemu z klientów zwracana jest BANKIEROWI dopiero w momencie zaspokojenia całkowitej POTRZEBY klienta (tzn. wówczas, gdy np. POŻYCZKA klienta A pokrywa POTRZEBĘ klienta A,) należy podać sposób (tzn. wielkości i kolejności) udzielania POŻYCZEK.

Pomijając oczywisty przypadek, w którym BANKIER udziela kolejnych POŻYCZEK w wysokości zgłaszanych przez klientów limitów POTRZEB, można rozważyć również inne warianty, w których BANKIER decyduje się udzielić limitowanych POŻYCZEK (tzn. mniejszych od zgłaszanych POTRZEB) wszystkim swoim klientom. Jeśli założymy, że w stanie S1 (**stan poprawny**, bo można zakończyć wszystkie zadania) przedstawionym na kolejnym rysunku klienci nie będą oddawać przydzielonego kapitału, to łatwo można zauważyć, że BANKIER może podjąć 8 różnych decyzji – dwie spośród nich odpowiadają sytuacjom S11 = S1_1 i S12 = S1_2. Decyzje te obejmują wszystkie możliwe przypadki rozdziału jednej lub dwóch jednostek waluty do klientów.

Przyjmując, że wektor [A; B; C; W] oznacza odpowiednio liczbę jednostek waluty przydzielonych klientowi A, B, C, oraz liczbę wolnych jednostek W, tj. dostępnych u BANKIERA, ze stanu S1 = [4; 2; 2; 2] można osiągnąć następujących 8 stanów: S2 = [5; 2; 2; 1], S3 = [4; 3; 2; 1], S4 = [4; 2; 3; 1] (przydział jednej jednostki), S5 = [5; 3; 2; 0], S6 = [5; 2; 3; 0], S7 = [4; 3; 3; 0], S8 = [6; 2; 2; 0], S9 = [4; 2; 4; 0] (przydział dwóch jednostek). W tym zestawieniu pominięto przydział [4; 4; 2; 0], gdyż maksymalna potrzeba procesu B wynosi 3, więc nie ma potrzeby przydzielać mu 4 jednostki waluty.

W pierwszym wypadku (S1_1) = S3 podjęta decyzja prowadzi do zaspokojenia POTRZEB wszystkich klientów (sytuacja S_A), w drugim natomiast (S1_2) = S7, po zaspokojeniu POTRZBY zgłaszanej przez klienta B, BANKIER znajduje się w sytuacji, w której nie może już dalej pełnić swoich funkcji (sytuacja S_B).

W rozważanym przykładzie POTRZEBY klientów interpretowane są jako operacje dla zaspokojenia, których niezbędne są pożyczki czyli zasoby. Jak łatwo zauważyć, nie każdy sposób udzielania pożyczek gwarantuje ich zwrot (jest bezpieczny). Pytanie jak uniknąć tego typu impasów (blokad) pozostawiamy chwilowo otwarte. Istotną w tym momencie jest sama refleksja związana z oceną złożoności problemu bezpiecznego (bez występowania impasów) i najkorzystniejszego (uwzględniającego np. wpływy z oprocentowania pożyczek) przydziału pożyczek (zasobów) do klientów (procesów).



Widać, że skutki rozstrzygnięć (np. ad hoc) konfliktowych żądań przez bankiera mogą się ujawnić dopiero po pewnym czasie. Z kolei eksplozja kombinatoryczna, towarzysząca przeglądowi wszystkich możliwych wariantów rozstrzygnięć konfliktów zasobowych, uniemożliwia ich wcześniejszą analizę.

W ogólnym przypadku odpowiedź na pytanie czy dany stan rozdziału zasobów jest bezpieczny, tzn. czy nie prowadzi do impasu (problem decyzyjny), jest **problemem NP-zupełnym** Oznacza to, że większość znanych w literaturze czasowo efektywnych algorytmów unikania impasów to rozwiązania heurystyczne, oparte na warunkach wystarczających do bezblokadowej realizacji zadań. Algorytmy heurystyczne mogą odrzucać niektóre, bezpieczne stany rozdziału zasobów i w konsekwencji prowadzić do zmniejszenia wydajności systemu.

Z przedstawionych rozważań wynika, że alternatywne algorytmy unikania blokad można porównywać stosując kryterium ilościowe jakim jest liczba akceptowanych stanów bezpiecznych. Dla określonego systemu procesów rozwiązanie jest tym lepsze im mniej odrzuca poprawnych stanów rozdziału zasobów.

W przedstawionych przykładach istniała również możliwość doprowadzenia wybranych procesów do zagłodzenia.

Zagłodzenie (ang. starvation) ma miejsce w przypadku, gdy pewien proces, na przykład z powodu posiadania niższego niż inne procesy priorytetu wykonania, nie może uzyskać dostępu do współdzielonego zasobu, lub też dostęp ten jest zbyt ograniczony. Zagłodzenie nie zatrzymuje systemu, ale może prowadzić do uciążliwych kłopotów, na przykład, jeśli zagłodzony zostanie proces, z którego efektów działania będą korzystać inne procesy. Korzystając raz jeszcze z analogii do ruchu ulicznego, sytuację zagłodzenia dobrze oddaje próba wjazdu samochodu z ulicy podporządkowanej na ruchliwą ulicę główną.

Aby wyeliminować zagłodzenie należy stosować priorytety dostępu do zasobów gwarantujące uzyskanie zasobu w skończonym czasie, np. FIFO, priorytet cykliczny.

6.3. Klasy problemów rozstrzygania konfliktów zasobowych

Problemy rozdziału zasobów w systemach procesów współbieżnych dotyczą głównie systemów operacyjnych, sieci komputerowych oraz systemów produkcyjnych, dla których rozpatrywane są zadania:

• Zapobiegania występowaniu zagłodzenia procesów (tj. sytuacji, w których dla pewnych procesów czasy oczekiwania na dostęp do zasobów są znacznie dłuższe niż czasy oczekiwania innych procesów, w praktyce procesy te nigdy nie uzyskują dostępu do zasobów).

• Zabezpieczania przed wystąpieniem stanów martwego punktu (tj. sytuacji, w których pewne procesy nie mogą dokończyć realizacji zadań, gdyż nie jest możliwe spełnienie ich żądań zasobowych).

• Wykrywania i likwidacji martwego punktu.

Rozwiązanie wymienionych problemów sprowadza się do konstrukcji odpowiedniego algorytmu rozstrzygania konfliktów zasobowych.

Zapobieganie zagłodzeniom

Zapobieganie występowaniu zagłodzeń procesów wymaga konstrukcji reguł alokacji zasobów, które gwarantują każdemu procesowi dostęp do żądanych zasobów w ciągu skończonego (ograniczonego z góry) okresu czasu. W praktyce jest to realizowane za pomocą reguł priorytetu opartych na priorytecie cyklicznym (prawo do korzystania z danego zasobu jest cyklicznie przekazywane procesom korzystającym z zasobu – zasada pierścienia logicznego) lub kolejce zgłoszeń typu FIFO – rozpatrywanie żądań zasobowych zgodnie z kolejnością pojawiania się żądań zasobowych procesów.

Zabezpieczanie przed blokadami

Zabezpieczanie przed blokadami sprowadza się do opracowania reguł przydziału zasobów, które nie dopuszczają do powstania stanu blokady. Wyróżnia się dwa podejścia do zabezpieczania przed blokadami: **zapobieganie** powstawaniu blokad oraz **unikanie blokad**.

W przypadku zapobiegania blokadom akceptowane są tylko takie żądania zasobowe procesów, których realizacja uniemożliwia spełnienie jednego z warunków koniecznych istnienia blokady (np. istnienia zamkniętego cyklu żądań zasobowych). W wielu przypadkach osiągnięcie stanu poprawnego w sensie danego algorytmu zapobiegania może wiązać się z koniecznością uwalniania zasobów przez procesy (np. algorytm liniowy Havendera – przydział zasobów do procesu zgodnie z rosnącą numeracją zasobów).

Natomiast unikanie blokad sprowadza się do wykonywania testu bezpieczeństwa stanów rozdziału zasobów, który polega na sprawdzeniu czy z danego stanu można osiągnąć, w wyniku stosowania określonych reguł alokacji zasobów, stan zakończenia wszystkich zadań (np. test bankiera).

W niektórych przypadkach inna reguła zapobiegania lub unikania blokad może stanowić test bezpieczeństwa stanów w algorytmie unikania blokad, np. reguła akceptująca tylko stany, w których liczba procesów wprowadzanych do systemu jest mniejsza niż długość najkrótszego cyklu w grafie żądań zasobowych procesów. W danym stanie sprawdzamy, czy istnieje ścieżka stanów, która prowadzi do stanu, w którym liczba procesów w systemie spełnia warunek minimalnego cyklu. Reguła ta może być traktowana jako statyczna reguła zapobiegania blokadom (np. zliczamy liczbę procesów w systemie i na tej podstawie wprowadzamy lub nie nowy proces) albo jako dynamiczna reguła unikania blokad, zależna od stanu.

Ze względu na NP-zupełność problemu unikania blokad w systemach procesów współbieżnych, w których procesy mogą zgłaszać częściowe żądania zasobowe, istniejące efektywne algorytmy unikania blokad są oparte na warunkach wystarczających do bezblokadowej realizacji zadań. Oznacza to, że mogą one odrzucać niektóre stany rozdziału zasobów, które nie prowadzą do blokady. Opracowane algorytmy różnią się liczbą akceptowanych stanów rozdziału zasobów oraz złożonością obliczeniową. Na tej podstawie można oceniać i porównywać jakość różnych algorytmów rozstrzygania konfliktów zasobowych.

Z kolei problem wykrywania i likwidacji blokad sprowadza się do wykrycia, które procesy i zasoby tworzą blokadę oraz opracowania procedury jej likwidacji.

Rodzaje problemów alokacji

W przypadku konkretnych systemów procesów współbieżnych są definiowane szczegółowe problemy rozstrzygania konfliktów zasobowych, które uwzględniają założenia i ograniczenia wynikające ze specyfiki ich działania. W większości systemów, w których występują problemy związane z istnieniem konfliktów zasobowych, procesy współbieżne realizują określone programy, które składają się z operacji wykonywanych z udziałem odnawialnych zasobów systemowych. Niektóre z tych zasobów mogą być wspólnie wykorzystywane przez procesy w trybie wzajemnego wykluczania, a w ramach operacji zasoby mogą być zarówno przydzielane do procesu, jak również zwalniane przez proces. Oznacza to, że podczas wykonywania programów procesy mogą zgłaszać częściowe żądania zasobowe.

W systemach operacyjnych struktura częściowych żądań zasobowych procesów (powiązania między operacjami procesów oraz zasoby niezbędne do wykonania operacji) nie są znane przed rozpoczęciem programów, gdyż wynika ona z dynamicznych interakcji procesów oraz działania użytkowników systemu komputerowego. Na przykład podczas realizacji programów mogą być znane jedynie maksymalne wartości, określające liczby zasobów niezbędnych do ich wykonania. Założenia takie są spotykane np. w klasycznym problemie bankiera, który sprowadza się do rozwiązania problemu blokad w systemach operacyjnych.

Z kolei w elastycznych systemach produkcyjnych (ESP) dla każdej marszruty technologicznej znana jest relacja następstwa operacji (porządek operacji), a także struktura zasobów (rodzaje i liczby jednostek) niezbędnych do wykonania każdej operacji. Podobnie jest w przypadku sieci komputerowych typu SFN (ang. Store-and-Forward Networks). W sieciach tego typu znana jest struktura tras przesyłania wiadomości oraz pojemności buforów (liczby jednostek zasobów), znajdujących się w poszczególnych węzłach sieci.

W obu przypadkach (ESP i SFN) uwzględnienie **dodatkowych informacji** o strukturze realizowanych programów pozwala konstruować algorytmy rozstrzygania konfliktów zasobowych, które są efektywniejsze w sensie wybranych kryteriów, np. liczby akceptowanych stanów rozdziału zasobów, stopnia wykorzystania zasobów, czasu oczekiwania procesów na dostęp do zasobów, niż algorytmy, które nie korzystają z takiej informacji.

Podsumowując, można stwierdzić, że wymienione problemy rozstrzygania konfliktów zasobowych: zabezpieczania przed zagłodzeniami i blokadami, oraz wykrywania i likwidacji blokad, uwzględniają założenia i ograniczenia wynikające ze specyfiki rozpatrywanych systemów procesów.

W szczególności, będą to algorytmy umożliwiające rozwiązywanie dwóch rodzajów problemów rozstrzygania konfliktów zasobowych:

• problemów, w których z założenia informacja o strukturze częściowych żądań zasobowych procesów nie jest dostępna, a więc nie jest ona wykorzystywana przez odpowiednie procedury alokacji zasobów (problemy klasy PR);

• problemów, w których przyjmuje się, że informacja o strukturze realizowanych programów – strukturze częściowych żądań zasobowych, jest znana przed rozpoczęciem procesów i wykorzystywana przez procedury alokacji zasobów (problemy klasy WPP).

Konkretne problemy są definiowane w oparciu o różne założenia i ograniczenia. Wiele z tych założeń jest wspólnych dla znacznej części problemów. Na podstawie analizy istniejącego stanu badań można przyjąć pewien zbiór założeń podstawowych Zp, które są najczęściej spotykane w rozpatrywanych zadaniach alokacji zasobów.

Założenia podstawowe Zp

Dla większości systemów procesów, rozpatrywanych w literaturze, istnieje pewien **wspólny** zbiór założeń i ograniczeń (Zp), definiujący określony problem rozstrzygania konfliktów zasobowych. Zbiór założeń Zp jest następujący:

• W systemie istnieje skończony zbiór zasobów odnawialnych R, które są wspólnie wykorzystywane przez skończony zbiór procesów P.

• Każdy zasób $r \in R$ posiada c(r) identycznych jednostek (zwanych pojemnością zasobu r), gdzie $c(r) \in N$ oraz N – zbiór liczb naturalnych.

• Jednostka zasobu $r \in R$ jest niepodzielna, tj. może ona być przydzielona w danej chwili tylko jednemu procesowi i to wtedy i tylko wtedy, gdy jest wolna (dostęp do jednostek zasobu odbywa się zgodnie z regułą wzajemnego wykluczania).

• Każdy proces polega na wykonaniu programu X składającego się ze skończonej liczby operacji, należących do zbioru $O(X) \subseteq OP$ i uporządkowanych przez relację częściowego, w szczególności liniowego porządku $Y \subseteq OPxOP$; gdzie
O(X) – zbiór operacji w programie X, natomiast OP – zbiór wszystkich operacji realizowanych w programach.

• Do wykonania każdej operacji $d \in OP$ niezbędny jest skończony, niepusty zbiór zasobów $R(d) \in R$ oraz $R(d) \neq \emptyset$ (w niektórych problemach zbiory te są znane i mogą być uwzględniane przez algorytmy rozstrzygania konfliktów zasobowych, w innych nie są z góry znane).

• Każda operacja korzysta z $i(d, r) \in N$ jednostek zasobu $r \in R(d)$, gdzie
$0 < i(d,r) \leq c(r)$.

• Dla każdej operacji $d \in OP$ określony jest czas jej wykonania $t(d) \in R+$, gdzie
$R+$ jest zbiorem liczb rzeczywistych dodatnich; wartość t(d) może być zmienną losową lub być wartością deterministyczną, reprezentującą średni czas wykonania operacji.

• Wykonanie kolejnej operacji $d \in OP$ przez proces $p \in P$ składa się z kilu etapów:

- żądania dostępu do zasobów R(d), niezbędnych do wykonania operacji d;
- zyskania dostępu do żądanych zasobów (rezerwacja zasobów);
- uwolnienia niektórych z posiadanych zasobów w momencie zarezerwowania zasobów R(d) (np. uwolnienie części zasobów, które nie są wykorzystywane w ramach kolejnej operacji d);
- uwolnienia posiadanych zasobów przez proces po zakończeniu aktualnie wykonywanej operacji i zajęciu zasobów niezbędnych do wykonania kolejnej operacji.

• Po zakończeniu operacji $d \in OP$ może być uwolnionych $0 < j(d,r) \leq i(d,r)$ jednostek zasobu $r \in U(d)$, gdzie $U(d) \subseteq R(d)$ (jeśli $U(d) = \emptyset$, to dwie kolejne operacje korzystają z tego samego zbioru zasobów).

• Liczba jednostek określonego zasobu, niezbędna do realizacji wszystkich procesów równocześnie, może być mniejsza od liczby wszystkich zasobów systemowych.

• Po zakończeniu programu (ostatniej operacji) proces uwalnia wszystkie przydzielone mu zasoby (zerowy bilans zasobowy).

Analiza własności systemów - wprowadzenie do klasyfikacji problemów

Z przedstawionych założeń wynika, że w zadaniach rozdziału zasobów spotykanych w systemach procesów współbieżnych każdy proces realizuje określony program, który składa się z operacji wykonywanych w oparciu o odpowiednie zasoby. W szczególności niektóre z zasobów mogą być współdzielone przez wiele procesów (wiele operacji), co może prowadzić do powstawania konfliktów zasobowych.

W praktyce każda operacja jest związana z dwoma etapami przejściowymi. Przed rozpoczęciem operacji realizowany jest etap rezerwacji zasobów, a po jej zakończeniu etap uwalniania zasobów. W szczególności możliwe są następujące przypadki:

• podczas wykonywania operacji proces może uwolnić część zasobów jeszcze przed dokonaniem rezerwacji zasobów niezbędnych do wykonania kolejnej operacji (spotykane w systemach komputerowych, np. proces zwalnia procesor, gdy oczekuje na urządzenie wejścia / wyjścia);

• podczas wykonywania operacji proces może uwolnić część zasobów w momencie dokonania rezerwacji zasobów niezbędnych do wykonania następnej operacji (spotykane w systemach ESP, w których każda operacja korzysta z kilku różnych zasobów, np. bufor maszyny jest zwalniany w momencie dokonania rezerwacji bufora wykorzystywanego podczas kolejnej operacji i rozpoczęcia operacji przenoszenia elementu między buforami przez roboty pomocnicze);

• proces uwalnia wszystkie zasoby wykorzystywane do realizacji określonej operacji po dokonaniu rezerwacji zasobów niezbędnych do wykonania kolejnej operacji i zakończeniu aktualnej operacji (np. spotykane w systemach ESP, w których każda operacja jest realizowana z wykorzystaniem jednego zasobu lub w sieciach komputerowych SFN, w których pakiety danych są przesyłane pomiędzy buforami znajdującymi się w węzłach sieci).

W zależności od specyfiki danego systemu procesów odpowiednie algorytmy rozstrzygania konfliktów zasobowych mogą korzystać z różnego rodzaju informacji dotyczących realizowanych programów.

W systemach komputerowych współbieżnie realizowane procesy mogą zgłaszać żądania zasobowe w sposób dynamiczny podczas realizacji programów, np. dynamiczny przydział bloków pamięci. Dlatego dokładna struktura częściowych żądań zasobowych nie jest znana przed rozpoczęciem programów.

W systemach tej klasy przyjmuje się, więc pewne maksymalne wartości żądań zasobowych, które mogą być zgłoszone przez proces (np. algorytm bankiera). Przydzielenie procesowi zasobów o wartościach równych żądaniom maksymalnym oznacza, że proces zakończy program w skończonym czasie i uwolni wszystkie przydzielone mu zasoby.

Z kolei w elastycznych systemach produkcyjnych (ESP) oraz w sieciach komputerowych przed rozpoczęciem programów znane są częściowe żądania zasobowe zgłaszane przez procesy podczas wykonywania określonych operacji. Dodatkowe informacje mogą być wykorzystane do konstrukcji bardziej efektywnych, np. w sensie liczby akceptowanych stanów rozdziału zasobów, algorytmów rozstrzygania konfliktów zasobowych.

Klasyfikacja problemów WPP

W przypadku zadań WPP, w których znana jest dokładna struktura żądań zasobowych w poszczególnych programach, można podzielić problemy WPP na podklasy w zależności od parametrów opisujących strukturę programów.

W szczególności można wyróżnić trzy parametry, umożliwiające definiowanie podproblemów w klasie zadań WPP:

- parametr O, opisujący porządek operacji w programach; może on przyjmować dwie wartości $o \in \{o_l, o_c\}$, gdzie: o_l – liniowy porządek operacji w programach; o_c – częściowy porządek operacji w programach (np. rozdzielanie i złączanie programów);

- parametr K, określający strukturę zbioru zasobów niezbędnych do wykonania operacji; może on przyjmować dwie wartości $K \in \{k_s, k_m\}$, gdzie: k_s – każda operacja korzysta z jednego rodzaju zasobu, tj. $(\forall d \in OP)(||R(d)|| = 1)$; k_m – każda operacja może korzystać z kilku rodzajów zasobów, należących do określonego zbioru zasobów, tj. $(\forall d \in OP)(||R(d)|| \geq 1)$; $||a||$ - liczba elementów zbioru a;

- parametr J, opisujący sposób pobierania jednostek zasobów danego rodzaju wykorzystywanych w ramach operacji; może on przyjmować dwie wartości $J \in \{j_u, j_g\}$, gdzie: j_u – każda operacja korzysta z jednostki każdego zasobu, tj. $(\forall d \in OP)(\forall r \in R(d))\ [i(d,r) = 1]$; j_g – każda operacja może korzystać z dowolnej liczby jednostek różnych zasobów, należących do określonego zbioru zasobów, tj. $(\forall d \in OP)(\forall r \in R(d))\ [i(d,r) \geq 1]$.

Wymienione założenia pozwalają zdefiniować w klasie zadań WPP osiem klas podproblemów, które w sposób symboliczny można opisać za pomocą notacji WPP|O,K,J, gdzie: $O \in \{o_l, o_c\}$, $K \in \{k_s, k_m\}$, $J \in \{j_u, j_g\}$.

W szczególności są to następujące klasy problemów rozstrzygania konfliktów zasobowych:

- WPP|o_l, k_s, j_u – klasa problemów, w których każdy program jest liniowo uporządkowanym ciągiem kolejnych operacji, a każda operacja korzysta z jednej jednostki określonego zasobu; jest to najprostsza i najbardziej powszechna klasa problemów; obejmuje ona problemy spotykane w sieciach komputerowych typu „store-and-forward” (SFN), w których pakiety są zapamiętywane w buforach, znajdujących się w węzłach sieci, należących do trasy przesyłania danych; problemy tej klasy są również rozpatrywane w kontekście systemów ESP oraz systemów komputerowych;

- WPP|o_l, k_s, j_g – klasa problemów, w których każdy program jest liniowo uporządkowanym ciągiem kolejnych operacji, a każda operacja może korzystać z kilku jednostek określonego zasobu; spotykane np. w systemach ESP, w których do wykonania danej operacji może być wymaganych kilka buforów maszyny lub kilka robotów;

- WPP|o_l, k_m, j_u – klasa problemów, w których każdy program jest liniowo uporządkowanym ciągiem kolejnych operacji, a każda operacja może korzystać z wielu różnych zasobów, ale tylko z jednej jednostki każdego zasobu; problemy spotykane w systemach ESP, w których operacje wymagają kilku różnych zasobów (np. dwóch różnych robotów oraz bufora maszyny);

- WPP|o_l, k_m, j_g – klasa problemów, w których każdy program jest liniowo uporządkowanym ciągiem kolejnych operacji, a każda operacja może korzystać z wielu różnych zasobów i dodatkowo z kilku jednostek określonych zasobów; problemy spotykane w systemach ESP, w których operacje mogą korzystać z kilku jednostek należących do różnych typów zasobowych (np. dwóch różnych robotów oraz kilku buforów maszyny);

- WPP|o_c, k_s, j_u – klasa problemów, w których każdy program jest częściwo uporządkowanym ciągiem operacji, a każda operacja korzysta z jednej jednostki określonego zasobu; jest to klasa problemów uwzględniająca możliwość łączenia się kilku programów w ramach jednej operacji lub rozdzielania się programu na kilka podprogramów; przypadki tego typu obejmują, np. operacje montażu i demontażu elementów występujące w systemach ESP;

- WPP|o_c, k_s, j_g – klasa problemów, w których każdy program jest częściowo uporządkowanym ciągiem operacji, a każda operacja może korzystać z kilku jednostek określonego zasobu; spotykane np. w systemach ESP, w których występują operacje montażu i demontażu elementów, a do wykonania danej operacji mogą być potrzebne, co najmniej dwa bufory maszyny lub dwa roboty.

- WPP|o_c, k_m, j_u – klasa problemów, w których każdy program jest częściowo uporządkowanym ciągiem operacji, a każda operacja może korzystać z wielu różnych zasobów, ale tylko z jednej jednostki każdego zasobu; problemy spotykane w systemach ESP, w których występują operacje montażu i demontażu elementów z udziałem kilku różnych zasobów (np. dwa roboty oraz bufor maszyny);

- WPP|o_c, k_m, j_g – klasa problemów, w których każdy program jest częściowo uporządkowanym ciągiem operacji, a każda operacja może korzystać z wielu różnych zasobów i dodatkowo z kilku jednostek określonych zasobów; problemy spotykane w systemach ESP, w których występują operacje montażu i demontażu elementów z udziałem kilku jednostek, należących do różnych typów zasobowych (np. dwóch różnych robotów oraz kilku buforów maszyny); jest to najbardziej ogólna klasa problemów, w której uwzględnia się pełną informację o strukturze żądań zasobowych procesów.

Należy podkreślić, że zadania klasy WPP są szczególnym przypadkiem zadań klasy PR, gdyż uwzględnia się w nich dodatkowe założenia określające strukturę programów. Wynika stąd, że rozwiązania opracowane dla zadań klasy PR mogą być wykorzystywane do rozwiązywania zadań klasy WPP, np. algorytm bankiera, który został opracowany dla zadań klasy PR, można wykorzystać do rozwiązywania zadań unikania blokad klasy WPP, wystarczy tylko obliczyć maksymalne żądania zasobowe dotyczące każdego zasobu, zgłaszane przez procesy podczas realizacji zadanych programów lub fragmentów programów, a następnie wykonywać przydziały zasobowe w taki sposób, aby spełniony był test bankiera.

Jeżeli dodatkowo uwzględni się, że liniowy porządek jest szczególnym przypadkiem częściowego porządku, pojedyncze zasoby na operację stanowią szczególny przypadek wielu zasobów na operację, a pojedyncze żądania zasobowe są odmianą wielokrotnych żądań zasobowych, to można stwierdzić, że problemy WPP|o_c,k_m,j_g tworzą najogólniejszą klasę zadań, natomiast problemy WPP|o_l,k_s,j_u klasę najbardziej szczegółową.

Wynika stąd, że algorytmy opracowane dla klasy WPP|o_c,k_m,j_g mogą być wykorzystywane do rozwiązywania wszystkich pozostałych zadań klasy WPP.

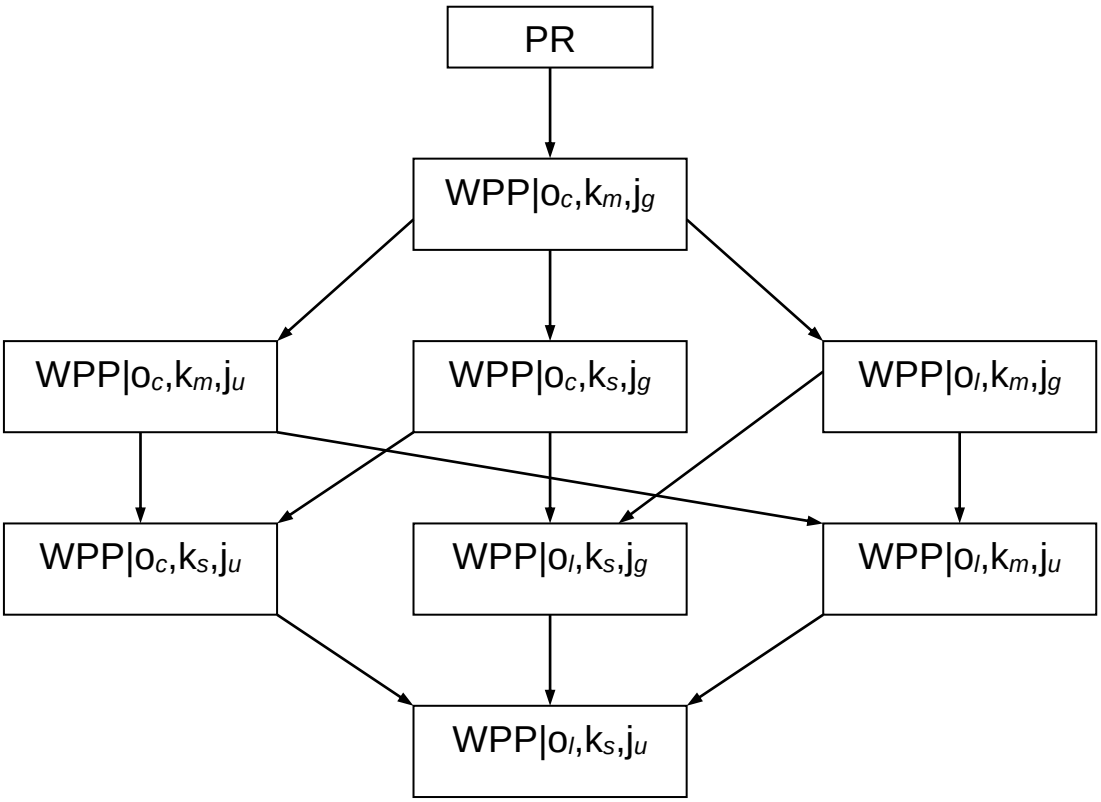
Jeśli przyjmie się, że zależność X $\subseteq$ Y oznacza, że problemy klasy X są podproblemami zadań klasy Y, to wówczas algorytmy rozstrzygania konfliktów zasobowych opracowane dla Y mogą być również wykorzystywane do rozwiązywania zadań klasy X. W szczególności, dla zdefiniowanych klas zadań spełnione są następujące relacje zawierania:

- WPP|o_c,k_m,j_g $\subseteq$ PR - problemy klasy WPP|o_c,k_m,j_g stanowią podklasę zadań PR i mogą być rozwiązywane za pomocą algorytmów opracowanych dla zadań PR, np. za pomocą algorytmu bankiera.

- WPP|o_l,k_s,j_u $\subseteq$ WPP|o_c,k_s,j_u $\subseteq$ WPP|o_c,k_m,j_u $\subseteq$ WPP|o_c,k_m,j_g oraz WPP|o_l,k_m,j_u $\subseteq$ WPP|o_c,k_m,j_u.

- WPP|o_l,k_s,j_u $\subseteq$ WPP|o_l,k_m,j_u $\subseteq$ WPP|o_l,k_m,j_g $\subseteq$ WPP|o_c,k_m,j_g oraz WPP|o_l,k_s,j_g $\subseteq$ WPP|o_l,k_m,j_g.

- WPP|o_l,k_s,j_u $\subseteq$ WPP|o_l,k_s,j_g $\subseteq$ WPP|o_c,k_s,j_g $\subseteq$WPP|o_c,k_m,j_g.



Założenia dodatkowe Zd

Z przeprowadzonych rozważań wynika, że problemy rozstrzygania konfliktów zasobowych (np. rozwiązywania problemu martwego punktu) posiadają pewien wspólny zbiór założeń podstawowych Zp.

W wielu przypadkach oprócz założeń podstawowych Zp przyjmowane są pewne założenia dodatkowe Zd, które uwzględniają specyficzne własności rozpatrywanych systemów procesów współbieżnych. Większość z tych założeń odzwierciedla sposób działania rzeczywistych systemów spotykanych w praktyce, np. systemów operacyjnych, komputerów równoległych, sieci komputerowych i systemów rozproszonych, baz danych oraz elastycznych systemów produkcyjnych i systemów zarządzania wózkami AGV. Założenia dodatkowe umożliwiają zdefiniowanie ostatecznej postaci rozpatrywanego problemu rozdziału zasobów, dla którego są poszukiwane odpowiednie algorytmy rozstrzygania konfliktów zasobowych, umożliwiające rozwiązanie problemu martwego punktu. Do najczęściej spotykanych założeń dodatkowych zalicza się:

- założenia dotyczące struktury zasobów (np. wszystkie zasoby są współdzielone, część zasobów nie jest współdzielona; pojemności wszystkich zasobów są co najmniej równe dwa, istnieją zasoby, których pojemność wynosi jeden);

- założenia określające strukturę żądań zasobowych, występujących w ramach realizowanych programów (np. operacje pobierania określonego zasobu poprzedzają wszystkie operacje uwalniania tego zasobu w danym programie; operacje mogą być realizowane z wykorzystaniem alternatywnych zbiorów zasobów, co odzwierciedla wielowariantowość i elastyczność przebiegu marszrut produkcyjnych w ESP lub możliwość wyboru alternatywnych tras przejazdu wózków transportowych AGV);

- założenia dotyczące sposobu realizacji procesów i metody rozwiązywania problemu blokad (np. czy istnieje możliwość odbierania zasobów procesom w celu likwidacji blokad lub spełnienia określonych statycznych reguł zapobiegania blokadom).

Rozwiązanie problemu rozstrzygania konfliktów zasobowych w systemie procesów współbieżnych sprowadza się do wyznaczenia reguł przydziału zasobów, gwarantujących bezblokadową realizację programów przez procesy. Wspomniane reguły umożliwiają rozstrzyganie konfliktów zasobowych w systemie procesów, w którym spełnione są określone założenia.

Przykładowe sformułowanie zadania alokacji może mieć następującą postać:

- podać reguły alokacji zasobów, gwarantujące bezblokadową realizację programów i umożliwiające optymalizację określonych wskaźników oceny efektywności funkcjonowania systemu procesów, w którym problem rozdziału zasobów jest definiowany za pomocą zbioru założeń podstawowych Zp oraz zbioru założeń dodatkowych Zd.

Założenia uwzględniające sposób realizacji procesów i metodę rozwiązywania problemu blokad określają, jakie podejścia do rozwiązywania problemu martwego punktu można stosować w danym systemie, np. czy można stosować podejścia, które zakładają możliwość wywłaszczania procesów z posiadanych zasobów w celu rozwiązania problemu blokad, czy też zastosowanie takiego podejścia nie jest możliwe.

Złożoność obliczeniowa problemów rozstrzygania konfliktów zasobowych

W literaturze problemu wykazano, że zadania alokacji klasy $WPP|o_l,k_s,j_u$ są problemami NP-zupełnymi. Na mocy zależności $WPP|o_l,k_s,j_u \subseteq WPP \subseteq PR$ problemy PR i WPP są w ogólnym przypadku zagadnieniami trudnymi obliczeniowo. W związku z tym, dla wielu z tych zadań nie należy spodziewać się istnienia optymalnych (tj. akceptujących wszystkie stany bezpieczne) i czasowo efektywnych algorytmów rozstrzygania konfliktów zasobowych. Wynika stąd, konieczność poszukiwania rozwiązań suboptymalnych, które mogą być wykorzystywane w systemach czasu rzeczywistego.

6.4. Metody i algorytmy rozwiązywania problemu martwego punktu

Znane w literaturze metody rozstrzygania konfliktów zasobowych umożliwiają rozwiązywanie różnych klas problemów (zobacz poprzedni punkt) i są implementowane w postaci odpowiednich algorytmów.

Istniejące metody (algorytmy) dzielą się na:

- metody (algorytmy) wykrywania i likwidacji blokad;
- metody (algorytmy) zapobiegania blokadom;
- metody (algorytmy) unikania blokad.

W przypadku konkretnych problemów definiowane są szczegółowe założenia i ograniczenia (podstawowe - Zp i dodatkowe – Zd), uwzględniające specyfikę systemów, dla których projektowane są określone algorytmy rozstrzygania konfliktów zasobowych.

6.4.1. Metody wykrywania i likwidacji impasów

Metody wykrywania i likwidacji blokad opierają się na okresowym sprawdzaniu, czy w systemie procesów nie pojawił się stan blokady. W przypadku wykrycia stanu blokady jest realizowany algorytm likwidacji, który polega na odłączaniu określonych zasobów od wstrzymanych procesów, aż do momentu, gdy kolejne uwolnienie zasobu doprowadzi do zlikwidowania stanu blokady.

Przykładem może być algorytm wykrywania cykli w grafie alokacji zasobowych i uwalniania zasobów przez procesy zajmujące zasoby należące do cyklu.

Wykorzystywanie podejścia opartego na wykrywaniu i likwidacji blokad jest w praktyce ograniczone do systemów, w których istnieje możliwość wywłaszczania niektórych zasobów przetrzymywanych przez procesy, powtarzania operacji przez procesy – po ponownym uzyskaniu dostępu do uwolnionych wcześniej zasobów, oraz w przypadku, gdy blokada zdarza się niezbyt często. Narzut czasowy metody jest zwykle dość duży, gdyż obejmuje czasy:

- wykonywania algorytmów wykrywania blokad,
- częściowego lub całkowitego zablokowania systemu od chwili wystąpienia stanu blokady do chwili jego wykrycia,
- wykonywania algorytmów likwidacji blokad,
- powtarzania wykonywania procesów, od których odłączono zasoby niewywłaszczalne w trakcie likwidacji blokady.

Konieczność powtarzania przerwanych operacji, a nawet całych programów, zbyt częste wykonywanie procedur wykrywania blokad lub zbyt długie pozostawanie procesów w stanie blokady prowadzi do wydłużenia czasu realizacji zadań, a tym samym do zmniejszenia efektywności działania systemu procesów.

W przypadku, gdy możliwe jest nałożenie pewnych ograniczeń na sposób przydzielania zasobów do procesów, lub przed rozpoczęciem działania procesów znane są zasoby żądane przez procesy podczas wykonywania określonych operacji, to zamiast wykrywania i likwidacji blokad można zastosować metody oparte o **zapobieganie** i **unikanie** blokad. Rozwiązania tego typu mogą prowadzić do zwiększenia efektywności działania systemu, gdyż nie dopuszczają do powstawania blokad.

6.4.2. Metody zapobiegania impasom

Zapobieganie powstawaniu blokad opiera się na wykorzystaniu takich zasad projektowania struktury systemu lub zasad obsługi żądań zasobowych procesów, które uniemożliwiają spełnienie, co najmniej jednego z **warunków koniecznych** istnienia blokady (w stanie blokady warunki te muszą być spełnione jednocześnie) podanych w pracach Coffmana:

• *wyłączności* (wzajemnego wykluczania) – w danej chwili jednostka zasobu może być wykorzystywana przez tylko jeden proces;

• *niewywłaszczalności* (szeregowania bez wywłaszczania) – zasoby przydzielone procesowi nie mogą mu być odebrane przed zakończeniem wykonywania określonej operacji, tj. zasoby przydzielane procesom nie podlegają wywłaszczaniu;

• *częściowego przydziału* (przetrzymywania i czekania) – istnieje proces, który przetrzymuje przydzielone mu i potrzebne w dalszym ciągu zasoby, a jednocześnie oczekuje na zwolnienie zasobów przetrzymywanych przez inne procesy;

• *czekania cyklicznego* – istnieje zamknięty łańcuch procesów, z których każdy oczekuje na zasoby przetrzymywane przez poprzednika w łańcuchu.

W szczególności, metody zapobiegania blokadom wykorzystują **statyczne reguły** obsługi żądań zasobowych procesów (ustalone w trybie off-line i niezależne od stanu systemu), które gwarantują, że wszystkie, osiągane stany systemu są bezpieczne, tj. nie prowadzą do blokady po skończonej liczbie przydziałów zasobowych.

Przed wykonaniem przydziału zasobów do procesu sprawdzany jest określony warunek zapobiegania blokadzie. Jeśli jest on spełniony, to przydział zasobów jest wykonywany, w przeciwnym przypadku proces czeka na dostęp do zasobów.

W niektórych systemach realizacja przydziałów zasobowych zgodnie ze statyczną regułą alokacji może wymagać, aby procesy uwalniały część z posiadanych zasobów, np. algorytm liniowy Havendera, oparty o zasadę zachowania liniowego porządku numerów zasobów przydzielanych do procesu.

Negacja warunku wyłączności

W celu zagwarantowania, że warunek wyłączności nie będzie nigdy spełniony, należy bądź zezwolić procesom na jednoczesny dostęp do zasobów systemu, bądź też zapewnić, że w trakcie realizacji procesy nie będą korzystały ze wspólnych zasobów, tzn. nie będą współzawodniczyły o dostęp do zasobów.

W praktyce warunek wzajemnego wykluczania procesów w dostępie do wspólnych zasobów stanowi podstawę działania wielu systemów procesów współbieżnych i dlatego nie jest możliwe jego wyeliminowanie.

Negacja warunku niewywłaszczalności

Negacja warunku niewywłaszczalności sprowadza się do wymuszenia czasowego zwolnienia zasobów przetrzymywanych przez procesy oczekujące na przydział innych zasobów, tj. do czasowego wywłaszczenia zasobów na rzecz innych procesów.

W szczególności zarządzanie zasobami powinno odbywać się zgodnie z następującym protokołem:

• jeśli proces wykorzystujący pewne zasoby oczekuje na spełnienie żądań zasobowych, które w danej chwili nie mogą być zrealizowane, wówczas zwalnia wszystkie aktualnie przydzielone mu zasoby;

• wywłaszczone zasoby są dopisywane do listy zasobów, na które proces oczekuje,

• proces jest wznawiany wówczas, kiedy wszystkie zasoby niezbędne do wykonania kolejnej operacji, stają się dostępne (zasoby odebrane procesowi oraz zasoby, na które oczekiwał).

W wielu systemach zanegowanie tego warunku nie jest możliwe ze względu na fizyczne ograniczenia, np. w elastycznych systemach produkcyjnych, w których wywłaszczanie procesów z zasobów oznaczałoby usuwanie elementów z buforów maszyn.

Negacja warunku częściowego przydziału

Dla zagwarantowania, że warunek częściowego przydziału nigdy nie będzie spełniony należy zapewnić, że żaden z procesów współbieżnych nie będzie oczekiwał na przydzielenie mu kolejnych, żądanych zasobów.

Odpowiednie algorytmy, gwarantujące spełnienie przedstawionej reguły, działają według zasady „wszystko albo nic”. Zgodnie z tą regułą przed rozpoczęciem pierwszej operacji procesowi muszą być przydzielane wszystkie zasoby, z których będzie korzystał podczas realizacji zadanego programu. Oznacza to, że procesy nie mogą w trakcie swojego wykonywania zgłaszać dalszych żądań zasobowych, mogą natomiast zwalniać niektóre z przydzielonych im zasobów.

Implementacja podejścia opartego na regule „wszystko albo nic” wykorzystuje koncepcję strefy krytycznej, w której zawarte są wszystkie zasoby współdzielone przez procesy. W danej chwili tylko jeden proces może korzystać z zasobów strefy krytycznej, tj. obsługa dostępu do strefy jest realizowana w oparciu o mechanizm wzajemnego wykluczania procesów.

Negacja warunku czekania cyklicznego

Metody oparte na zaprzeczeniu warunku czekania cyklicznego wykorzystują zasadę liniowego uporządkowania numerów wszystkich zasobów przydzielanych każdemu procesowi.

Stosowanie tej zasady uniemożliwia powstanie cyklu żądań zasobowych procesów w grafie alokacji zasobów, opisującym dowolny stan rozdziału zasobów w systemie procesów współbieżnych.

Klasycznym rozwiązaniem, wykorzystującym omawiane podejście, jest wspomniany już algorytm Havendera, w którym zasobom przypisuje się unikalne numery n(r1), n(r2), ..., n(rk), gdzie n(x) jest funkcją numeracji zasobów, która każdemu zasobowi rk przypisuje liczbę naturalną.

Żądania zasobowe procesów są realizowane w taki sposób, aby numery kolejnych zasobów przydzielanych każdemu procesowi tworzyły ciąg monotoniczny.

W omawianej metodzie dopuszcza się jednak możliwość odbierania zasobów określonym procesom w przypadku, gdy nie jest możliwe wykonanie przydziału zgodnie z zadaną regułą. Dlatego bez odpowiednich modyfikacji nie może ona być wykorzystywana do rozwiązywania problemów, w których struktura programów jest dowolna, a odbieranie zasobów procesom nie jest dozwolone (np. problemy występujące w elastycznych systemach produkcyjnych (ESP).

Główną zaletą metod zapobiegania przed blokadami jest to, że wykorzystują one statyczne reguły alokacji zasobów, które nie wymagają informacji o stanie systemu.

6.4.3. Metody unikania impasów

Unikanie blokad polega na analizie stanu systemu i zarządzaniu przydziałami zasobowymi w taki sposób, aby eliminowane były stany prowadzące do blokady. Jest to podejście dynamiczne, w którym wykorzystuje się informację o aktualnym stanie systemu oraz przyszłym zachowaniu procesów w celu eliminacji stanów rozdziału zasobów, dla których spełnione są równocześnie warunki konieczne dla istnienia blokady.

W celu realizacji unikania blokad wykorzystywana jest procedura analizy bezpieczeństwa stanów systemu, która korzystając z informacji o przyszłych żądaniach zasobowych procesów sprawdza czy istnieje sekwencja przydziałów zasobowych prowadząca do zakończenia wszystkich zadań.

Jednym ze sposobów unikania blokad jest przegląd zupełny przestrzeni stanów. W ogólnym przypadku jest to jednak podejście nieefektywne, gdyż problem unikania blokad jest problem trudnym obliczeniowo. Wynika stąd, że stwierdzenie czy dany stan rozdziału zasobów nie prowadzi w przyszłości do blokady może wymagać analizy wykładniczej liczby stanów.

W praktyce stosowane metody unikania blokad wykorzystują dodatkową informację mówiącą o sposobach realizacji żądań zasobowych procesów na podstawie, których definiowane są warunki wystarczające do unikania blokad.

Różnorodność odpowiednich warunków wynika z możliwości formułowania różnych założeń określających sposoby zgłaszania i realizacji żądań zasobowych. Jednym z najbardziej znanych testów jest test oparty na metodzie bankiera, w której przyjmuje się, że znane są maksymalne żądania zasobowe procesów dotyczące wszystkich zasobów i akceptuje tylko takie stany rozdziału zasobów, w których zawsze istnieje możliwość zakończenia wszystkich procesów.

Metody unikania blokad realizują każde żądanie przydziału zasobu, jeżeli związane z nim przejście stanów jest bezpieczne. Oznacza to, że kluczowymi dla omawianego podejścia są pojęcia:

- stanu bezpiecznego,

- bezpiecznej sekwencji przydziałów zasobowych,

- stanu niebezpiecznego,

- stanu blokady całkowitej.

Stan początkowy systemu (oznaczany S0) jest stanem bezpiecznym. W tym stanie żaden proces nie korzysta ze wspólnych zasobów. Można, więc zrealizować wszystkie zadania wprowadzając kolejno po jednym zadaniu do systemu (zakładamy, że zasoby dostępne w systemie są wystarczające do wykonania każdego zadania osobno).

Ogólna definicja stanu bezpiecznego jest następująca: stan systemu nazywa się bezpiecznym wtedy i tylko wtedy, gdy można osiągnąć z tego stanu, po skończonej liczbie przydziałów zasobów do określonych procesów, stan końcowy (SK = S0), w którym wszystkie procesy są zakończone (oznacza to, że w przestrzeni stanów istnieje przejście prowadzące od danego stanu do stanu końcowego, który z punktu widzenia stanu rozdziału zasobów jest równoważny stanowi początkowemu).

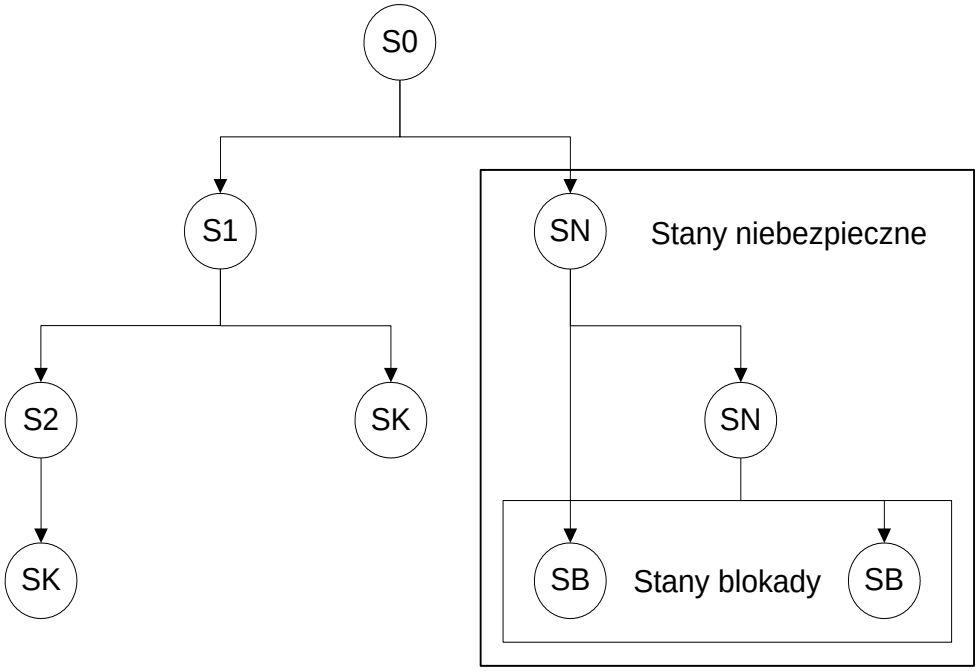
Bezpieczną sekwencją przydziałów zasobowych jest taka sekwencja, która zapewnia przeprowadzenie systemu z jednego stanu bezpiecznego do innego.

Stan systemu nazywa się **niebezpiecznym** wtedy i tylko wtedy, gdy z tego stanu nie jest możliwe osiągnięcie stanu końcowego, a po skończonej liczbie przydziałów zasobowych następuje przejście do stanu blokady (tj. ze stanu niebezpiecznego można przejść tylko do innego stanu niebezpiecznego lub do stanu blokady).

Stan systemu nazywa się stanem **blokady** wtedy i tylko wtedy, gdy istnieje zbiór procesów, dla których żądania zasobowe żadnego z nich nie mogą zostać spełnione.

Na rysunku pokazano strukturę przestrzeń stanów dowolnego systemu procesów współbieżnych konkurujących o dostęp do wspólnych zasobów.

W szczególności na rysunku poniżej przedstawiono stany niebezpieczne, stany blokady oraz stany bezpieczne. Stan początkowy oznaczony został, jako S0, kolejne stany bezpieczne, jako S1, S2 i SK - stan końcowy, przy czym SK jest faktycznie równy S0. Stany blokady oznaczone zostały, jako SB, natomiast stany niebezpieczne, jako SN.



Algorytmy oparte na warunkach wystarczających do zabezpieczania przed blokadami generują różne przestrzenie stanów. Mogą one w ogólnym przypadku odrzucać niektóre z poprawnych stanów rozdziału zasobów (algorytmy suboptymalne). Algorytm jest tym lepszy im większa jest uzyskiwana za jego pomocą przestrzeń stanów bezpiecznych danego systemu procesów.

Metody unikania blokad zapewniają najwyższy stopień wykorzystania zasobów systemu. Różnią się między sobą sposobami testowania żądań przydziału zasobów oraz algorytmami testów, charakteryzującymi się, na ogół, różną złożonością obliczeniową (np. algorytm bankiera, stref synchronizacji).

6.4.4. Inne rodzaje algorytmów

Inne kryterium na podstawie, którego można dokonać klasyfikacji algorytmów rozstrzygania konfliktów zasobowych uwzględnia stan rozdziału zasobów oraz sposób podejmowania decyzji o przydziałach zasobowych.

Na tej podstawie wyróżnia się:

- *algorytmy scentralizowane*, w których decyzje o przydziałach zasobowych są podejmowane na podstawie znajomości globalnego stanu rozdziału zasobów (znana jest aktualna informacja o zasobach przydzielonych wszystkim procesom); w szczególności istnieje nadrzędny proces decyzyjny, który zarządza przydziałami zasobowymi lub istnieje mechanizm komunikacji (protokół) umożliwiający zgromadzenie informacji o stanie globalnym systemu przez dowolny z procesów współbieżnych, które konkurują w dostępie do wspólnych zasobów (np. mechanizm przekazywania danych oparty o pierścień lub drzewo procesów);

- *algorytmy rozproszone*, w których podejmowanie decyzji o przydziałach zasobowych nie wymaga znajomości globalnego stanu rozdziału zasobów (znana jest aktualna informacja o stanie niektórych zasobów i procesów, np. o stanie zajętości buforów maszyn, między, którymi jest przewożony element).

Wymienione klasy algorytmów mogą być stosowane, zarówno w systemach o strukturze scentralizowanej, w których istnieje wspólna pamięć wykorzystywana do komunikacji między procesami (np. w oparciu o zmienne stanu, semafory, monitory, kolejki komunikatów), jak i w systemach rozproszonych, w których brak jest wspólnej pamięci, a procesy komunikują się w oparciu o przesyłanie wiadomości za pośrednictwem odpowiedniego medium transmisji danych.

Większość spotykanych w literaturze algorytmów zabezpieczania przed blokadami to rozwiązania czasowo efektywne, oparte na warunkach wystarczających do bezblokadowej realizacji procesów. Spełnienie takich warunków gwarantuje, że w systemie procesów na pewno nie będą osiągane stany rozdziału zasobów, które prowadzą do blokady. Odpowiednie algorytmy zabezpieczania mogą jednak odrzucać pewne stany poprawne, tj. nie prowadzące do blokady (tzw. stany bezpieczne). Konsekwencją odrzucania w systemie procesów poprawnych stanów rozdziału zasobów może być zmniejszenie efektywności działania systemu w sensie określonego kryterium, np. liczby możliwych do zrealizowania harmonogramów, czasu realizacji zadań.

Z przedstawionych rozważań wynika możliwość podziału algorytmów zabezpieczania przed blokadami na:

- *algorytmy optymalne* (maksymalnie dopuszczające), które odrzucają wyłącznie stany rozdziału zasobów prowadzące do blokady (akceptują wszystkie stany poprawne z punktu widzenia bezblokadowej realizacji procesów, tzw. stany bezpieczne); podstawę do konstrukcji takich algorytmów stanowią warunki konieczne i wystarczające do bezblokadowej realizacji procesów;

- *algorytmy suboptymalne*, które mogą eliminować niektóre stany poprawne (bezpieczne) z punktu widzenia bezblokadowej realizacji procesów; podstawę do konstrukcji takich algorytmów stanowią warunki wystarczające do bezblokadowej realizacji procesów.

6.5. Przegląd wybranych rozwiązań

W literaturze znanych jest wiele algorytmów umożliwiających rozwiązanie problemu martwego punktu w systemach procesów współbieżnych. Dotyczą one przede wszystkim systemów operacyjnych, komputerów równoległych i baz danych, sieci komputerowych, oraz elastycznych systemów produkcyjnych.

Istniejące podejścia wykorzystują różne modele, formalizmy i metody umożliwiające rozwiązywanie problemu blokad. Wśród najważniejszych wyróżnić można:

- metody oparte o wykorzystanie mechanizmów sterowania współbieżnością, gwarantujące wzajemne wykluczanie procesów w dostępie do wspólnych zasobów tworzących strefę krytyczną (metody typu „wszystko albo nic”);

- metody wykorzystujące modele algebraiczne: macierze stanu rozdziału zasobów i test bankiera, równania algebraiczne i macierzowe, zasadę liniowej numeracji zasobów lub znaczników czasowych przypisanych procesom;

- metody oparte o rachunek predykatów, wykorzystanie algebry procesów współbieżnych, rozproszonych, sekwencyjnych, oraz logikę temporalną;

- metody oparte o wykorzystanie grafów skierowanych;

- metody oparte o wykorzystanie systemów zdarzeniowych, automatów, sieci Petriego; w tym oparte o konstrukcję grafu stanów osiągalnych sieci; konstrukcję sieci wolnych od blokad dzięki eliminacji stanów typu „martwy syfon” w oparciu o programowanie całkowitoliczbowe, dobór dodatkowych miejsc kontrolnych, lub zastosowanie modeli macierzowych i grafowych; sterowanie przepływem znaczników w oparciu o test bankiera, strefy synchronizacji; projektowanie sieci Petriego i automatów, których grafy stanów osiągalnych mają zadaną strukturę i składają się ze stanów wolnych od blokad, np. w oparciu o teorię regionów, programowanie liniowe i całkowitoliczbowe;

- metody oparte o wykorzystanie programowania z ograniczeniami (CP) lub programowania logicznego z ograniczeniami (CLP), w których wyznaczanie sekwencji bezpiecznych przydziałów zasobowych lub wyznaczanie bezblokadowych harmonogramów sprowadza się do rozwiązywania określonych problemów spełnienia ograniczeń PSO, w których zmienne problemu są liczbami całkowitymi należącymi do określonych przedziałów wartości.

W zależności od zastosowań są to algorytmy scentralizowane lub rozproszone.

Algorytmy scentralizowane i rozproszone

W scentralizowanych systemach komputerowych procesy współbieżne są realizowane pod kontrolą jednego systemu operacyjnego i komunikują się w oparciu o mechanizmy wspólnej pamięci (semafory, monitory, komunikaty). W tym przypadku stosowane algorytmy rozstrzygania konfliktów zasobowych w systemach procesów współbieżnych (np. synchronizacji dostępu do sekcji krytycznej) mają charakter scentralizowany, gdyż korzystają z informacji o stanie globalnym systemu procesów.

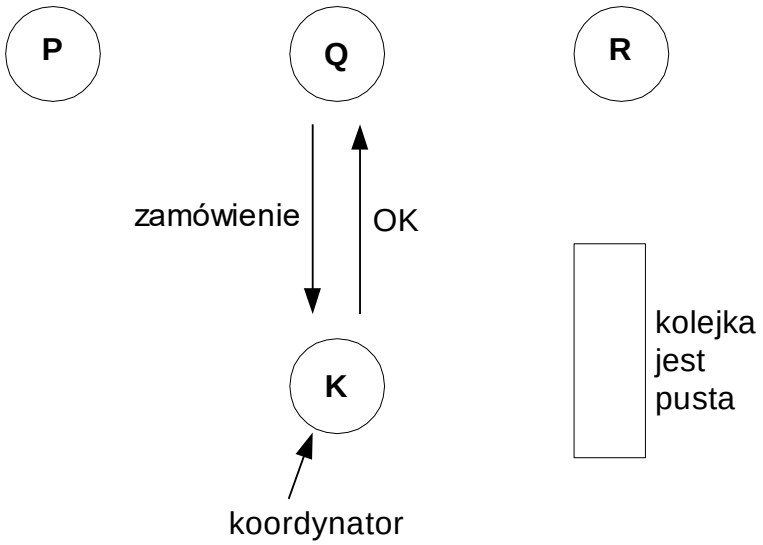
Z kolei algorytmy rozstrzygania konfliktów zasobowych (np. synchronizacji dostępu do sekcji krytycznej) w systemach rozproszonych procesów współbieżnych, mogą charakteryzować się podejściem rozproszonym lub scentralizowanym.

6.5.1. Algorytmy scentralizowane

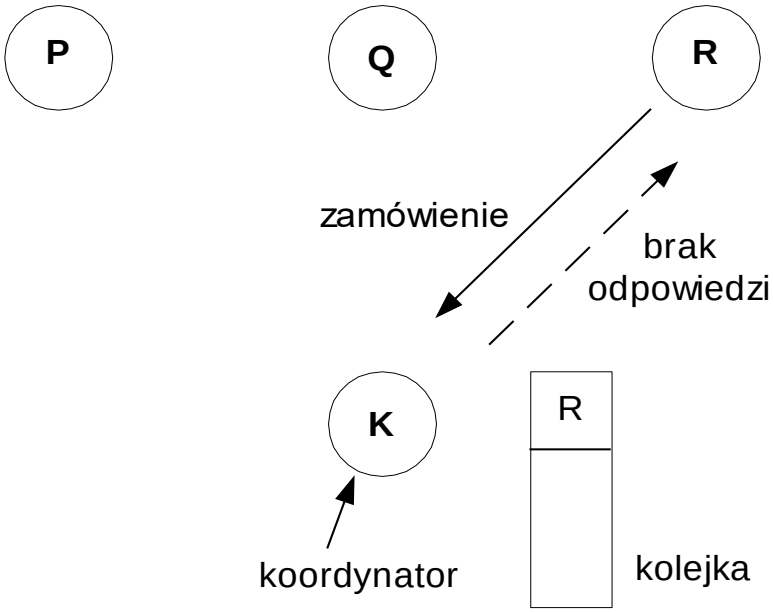
W systemie procesów rozproszonych podejście scentralizowane charakteryzuje się istnieniem pewnego procesu – koordynatora, który posiada pełny obraz sytuacji (zna stan globalny systemu) i w zależności od tego, np. czy sekcja krytyczna jest wolna, czy nie, może udzielić danemu procesowi pozwolenia na zajęcie zasobu (np. sekcji krytycznej) lub nie.

Rozproszone procesy współbieżne – synchronizacja scentralizowana

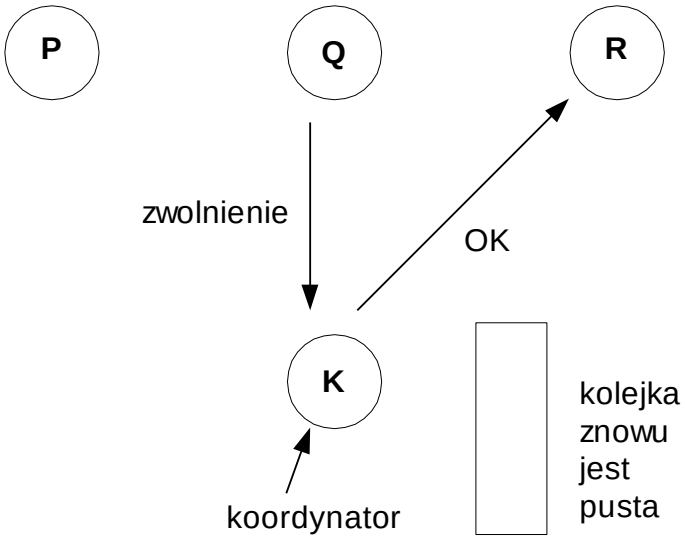
Proces koordynujący zarządza dostępem procesów do sekcji krytycznej, realizując zadania podobne do działań podejmowanych przez scentralizowany system komputerowy. Każdy proces żądający dostępu do sekcji krytycznej, wysyła odpowiedni komunikat z zamówieniem do koordynatora. Po odebraniu zgłoszenia koordynator sprawdza, czy w danej sekcji nie przebywa inny proces. Jeśli sekcja jest wolna, to natychmiast odsyła procesowi zgodę na jej zajęcie. Po otrzymaniu zezwolenia od koordynatora proces wchodzi do sekcji krytycznej i podejmuje działania w oparciu o dostępne zasoby.



W przypadku, gdy do koordynatora zgłosi się kolejny proces (np. R) podczas, gdy w sekcji krytycznej znajduje się jeszcze proces Q, koordynator nie odpowiada i umieszcza zamówienie procesu R w kolejce typu FIFO.



Po wyjściu z sekcji krytycznej proces Q wysyła do koordynatora komunikat informujący o zwolnieniu sekcji a następnie przechodzi do wykonywania kolejnych operacji. Wówczas koordynator pobiera z kolejki następne zamówienie (według ustalonego algorytmu, np. FIFO – ang. First In First Out) i wysyła potwierdzenie do procesu zamawiającego. W przedstawionym przykładzie jest to proces R, który dopiero po odebraniu zezwolenia może wejść do sekcji krytycznej.



Algorytm scentralizowany zapewnia wzajemne wyłączanie w dostępie do wspólnych zasobów, gdyż koordynator zezwala na wejście do sekcji krytycznej tylko jednemu procesowi. Algorytm ten jest ponadto sprawiedliwy (udziela pozwoleń w kolejności ich zamawiania), dzięki czemu nie występuje głodzenie procesów. Opisany schemat jest w miarę prosty, gdyż wymaga tylko trzech komunikatów do obsługi dostępu do sekcji krytycznej: zamówienia, odpowiedzi i zwolnienia. Ma on jednak pewne wady, charakterystyczne dla każdego scentralizowanego podejścia do problemu. W przypadku awarii koordynatora załamaniu ulega cały system. Ponadto, proces – koordynator może stać się wąskim gardłem w systemie, w którym musi zarządzać wieloma procesami.

Przykładem rozwiązania stosowanego w podejściu scentralizowanym jest algorytm Bankiera, w którym centralny koordynator realizuje przydziały zasobowe zgodnie ze schematem bankiera, który zakłada, że po przydzieleniu pewnego zbioru zasobów do procesu, osiągany stan jest poprawny (tj. nie prowadzi do blokady), jeśli w tym stanie możliwe jest spełnienie maksymalnych żądań zasobowych każdego z procesów.

Algorytm bankiera

Założenia:

- dla każdego procesu $p_i \in P$ znane są maksymalne żądania zasobowe dotyczące każdego zasobu $r_j \in R$,

- proces, który otrzyma wszystkie żądane zasoby zakończy działanie (uwolni przydzielone mu zasoby) w skończonym czasie,

- istnieje centralny proces, który tworzy kolejkę żądań zasobowych procesów i podejmuje decyzje o przydziałach zasobowych.

Przed przydzieleniem zasobu do procesu sprawdzane są warunki wystarczające dla uniknięcia impasu. Niech

$max_i(r)$ - maksymalna liczba jednostek zasobu $r \in R$ wykorzystywana przez proces $p_i \in P$,

$M(i) = [max_i(r_1), \ldots , max_i(r_n)]$ - wektor maksymalnych żądań zasobowych procesu $p_i \in P$,

$a_i(r)$ - liczba jednostek zasobu $r \in R$ wykorzystywana przez proces $p_i \in P$,

$f(r)$ - liczba dostępnych jednostek zasobu $r \in R$.

Załóżmy, że proces p_i do wykonania pewnej operacji żąda łącznie przydziału d jednostek zasobu r.

Test bankiera

Przydział zasobów do procesów jest poprawny (nie prowadzi do blokady) jeżeli spełnione są równocześnie następujące warunki:

- $a_i(r) + f(r) \geq d$, tzn. liczba jednostek zasobu r, które proces p_i może wykorzystać jest nie mniejsza od liczby jednostek żądanych przez proces,

- przydzielenie d jednostek zasobu r do procesu p_i prowadzi do stanu, w którym istnieje sekwencja przydziałów zasobowych umożliwiająca spełnienie *maksymalnych* żądań zasobowych wszystkich procesów, tzn. zbiór procesów można uporządkować w taki sposób, aby każdy proces uzyskał dostęp do zasobów określonych przez wektor żądań maksymalnych M(i).

Z przedstawionych reguł wynika, że w każdym kroku alokacji „bankier" zakłada najgorszy przypadek - zgłoszenie przez wszystkie aktywne procesy maksymalnych żądań zasobowych, i stara się znaleźć takie uporządkowanie procesów, które zapewni spełnienie ich maksymalnych żądań zasobowych.

Przykład.

W celu ilustracji działania algorytmu bankiera rozpatrzmy następujący system procesów.

p_1 : $\langle$ (1r), (3r), (4r) $\rangle$ - program nr 1,

p_2 : $\langle$ (1r), (3r), (5r) $\rangle$ - program nr 2.

Maksymalne żądania zasobowe procesów: $max_1(r) = 4$, $max_2(r) = 5$.

Pojemność zasobu: $C(r) = 5$.

Załóżmy, że w pewnej chwili czasowej pojawiła się następująca sekwencja żądań zasobowych procesów:

Sek = $\langle p_1 , p_2 , p_2 , p_1 \rangle$ - kolejka procesów,

Sres = (1, p_1, 1r), (2, p_2, 1r), (3, p_2, 3r), (4, p_1, 3r) - żądane zasoby.

Na mocy testu bankiera:

- żądania zasobowe o numerach 1, 2 i 4 zostaną zrealizowane,
- żądanie zasobowe numer 3 nie zostanie spełnione.

Dowód:

Alokacja 1: $(1, p_1, 1r)$; po alokacji $a_1(r)=1$, $f(r)=4$.

W wyniku alokacji zostaną wolne 4 jednostki zasobu r. Stan osiągnięty w wyniku alokacji jest bezpieczny, gdyż p_1 będzie mógł zakończyć działanie nawet, jeśli w następnym kroku zażąda dostępu do 4 jednostek zasobu.

Alokacja 2: $(2, p_2, 1r)$; po alokacji $a_1(r)=1$, $a_2(r)=1$, $f(r)=3$.

W wyniku drugiej alokacji pojawia się stan, w którym każdy z procesów korzysta z jednostki zasobu r i w systemie pozostają jeszcze 3 wolne jednostki. Jeżeli w kolejnym kroku procesy zgłoszą maksymalne żądania zasobowe, to mogą one być zrealizowane zgodnie z sekwencją p_1, p_2. Żądania procesu p_1 mogą być spełnione, gdyż $max_1(r) = 4 = a_1(r) + f(r)$, po wykonaniu operacji p_1 zwalnia 4 jednostki zasobu r, tj. $f(r) = 4$, i wówczas $max_2(r) = 5 = a_2(r) + f(r)$, a więc można zrealizować maksymalne żądania zasobowe procesu p_2. Stan rozdziału po dwóch alokacjach zostanie uznany za bezpieczny.

W stanie, w którym $a_1(r)=1$, $a_2(r)=1$, $f(r)=3$ nie jest natomiast możliwy przydział zasobów zgodnie z sekwencją p_2, p_1. W tym przypadku jako pierwsze zgłaszane są maksymalne żądania zasobowe procesu p_2, które wynoszą $max_2(r) = 5 > a_2(r) + f(r) = 4$. Sekwencja przydziałów p_2, p_1 nie zostanie uznana za prowadzącą do stanu bezpiecznego.

Na tym etapie zrealizowano żądania zasobowe zgodnie z przyjętą sekwencją żądań p_1, p_2. Spełnienie kolejnego żądania zasobowego zgłaszanego ponownie przez p_2 prowadzi do:

Alokacja 3: $(3, p_2, 3r)$; po alokacji $a_1(r)=1$, $a_2(r)=3$, $f(r) = 1$.

W osiągniętym stanie nie istnieje „bezpieczna” sekwencja przydziałów zasobowych, gdyż do spełnienia maksymalnych żądań zasobowych procesu p_1 niezbędne są 3 jednostki zasobu, natomiast do spełnienia maksymalnych żądań zasobowych procesu p_2 niezbędne są 2 jednostki zasobu.

Wniosek: alokacja 3 nie zostanie zrealizowana; żądanie procesu p_2 musi czekać na realizację.

Alokacja 4: (4, p_1, 3r); po alokacji $a_1(r)=3$, $a_2(r)=1$, $f(r) = 1$.

„Bezpieczna” sekwencja przydziałów zasobowych: p_1, p_2.

Złożoność obliczeniowa algorytmu bankiera:

$O(n^2)$ - dla n procesów i jednego zasobu,

$O(mn^2)$ - dla n procesów i m zasobów.

6.5.2. Algorytmy rozproszone

Rozproszone procesy współbieżne – synchronizacja rozproszona

W podejściu rozproszonym nie ma wyszczególnionego centrum sterowania. Dlatego algorytmy rozproszone muszą bazować na innych przesłankach niż wiedza o stanie całego systemu, np. związanych:

- z wymogiem całkowitego uporządkowania czasowego wszystkich zdarzeń w systemie (np. opartego na wymianie wiadomości pomiędzy procesami i przekazywaniu znaczników czasowych opartych o zegary logiczne), w taki sposób, aby ich kolejność była zawsze jednoznacznie określona.

- ze sposobem uporządkowania wykonania procesów, np. korzystając ze struktury pierścienia z żetonem (ang. token ring), w którym pomiędzy procesami przesyłany jest pewien znacznik (żeton) tworząc zamknięty pierścień. Tylko proces, który jest w posiadaniu żetonu może wejść do sekcji krytycznej. Po wyjściu z sekcji przekazuje on żeton dalej.

Algorytm porządkowania według znaczników czasu

Przykładem może być algorytm synchronizacji dostępu do strefy krytycznej w systemie, w którym wszystkie procesy są równouprawnione a dostęp do sekcji krytycznej uzyskuje proces, który został uzgodniony w wyniku negocjacji między procesami.

W omawianym algorytmie synchronizacji proces, który chce wejść do sekcji krytycznej wysyła do wszystkich procesów komunikat zawierający nazwę sekcji, do której chce wejść, swój numer (numer procesu) oraz znacznik czasu (bieżący czas). Ponadto przyjmuje się, że każdy komunikat jest potwierdzany (przy założeniu niezawodnej komunikacji). Decyzja procesu otrzymującego komunikat - zamówienie od innego procesu zależy od jego zamiarów względem wymienionej sekcji krytycznej. Można wyróżnić trzy przypadki:

1. Jeśli odbiorca nie jest w sekcji krytycznej i nie zamierza do niej wchodzić, wówczas natychmiast odsyła komunikat OK natychmiast.

2. Jeśli odbiorca właśnie przebywa w podanej sekcji krytycznej, to ustawia zamówienie w kolejce i opóźnia odpowiedź.

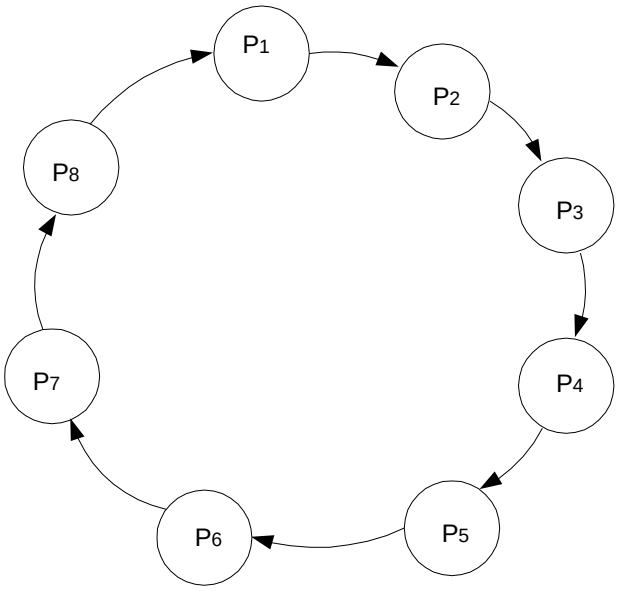
3. Jeśli odbiorca chce wejść do sekcji krytycznej, lecz jeszcze tego nie zrobił, to porównuje znacznik czasu odebranego komunikatu z tym, który sam wysłał w komunikacie do wszystkich procesów. Jeśli odebrany komunikat ma mniejszy znacznik czasu, to odbiorca odsyła komunikat OK, natomiast jeśli znacznik komunikatu jest większy (późniejszy niż znacznik własnego komunikatu z żądaniem dostępu do strefy krytycznej), to odbiorca ustawia zamówienie w kolejce i wstrzymuje odpowiedź.

Proces może wejść do sekcji krytycznej tylko wówczas, gdy otrzyma potwierdzenie od wszystkich procesów, do których wysłał komunikat. Gdy już opuści sekcję krytyczną wysyła komunikat potwierdzający OK do wszystkich procesów, które w tym czasie się do niego zgłosiły i usuwa ich zamówienia z kolejki.

Algorytm rozproszony zapewnia wzajemne wyłączanie procesów w dostępie do wspólnych zasobów, jak również eliminuje blokady i zagłodzenia. Wejście do sekcji krytycznej jest planowane na podstawie wartości znaczników czasu. Takie uporządkowanie gwarantuje obsługę procesów w kolejkach zgłoszeń w kolejności zgodnej z regułą FIFO.

Algorytm pierścienia z żetonem

Kolejną metodą zapewniającą wzajemne wykluczanie procesów w dostępie do wspólnych zasobów jest algorytm pierścienia z żetonem. W algorytmie tym procesy są łączone za pomocą oprogramowania w logiczną strukturę pierścienia.



Kolejność procesów w pierścieniu ustalana jest dowolnie, np. w numerycznym porządku adresów sieciowych. Sposób, w jaki porządkowane są procesy nie ma znaczenia. Ważne jest tylko to, aby każdy proces znał identyfikator swojego następnika w pierścieniu.

Żeton jest niczym innym jak specjalnym komunikatem przekazywanym w obrębie systemu. Proces, który otrzyma żeton sprawdza, czy nie ma zamiaru wejść do sekcji krytycznej. Jeśli tak to wchodzi do niej przechowując żeton, a po wyjściu z sekcji przesyła go do następnego procesu w systemie. W przeciwnym przypadku przekazuje go od razu do swego następnika w pierścieniu. Przy braku procesów żądających dostępu do sekcji krytycznej żeton krąży w pierścieniu. Dzięki temu, że w systemie występuje tylko jeden żeton, w danej chwili do sekcji krytycznej może wejść tylko proces, który jest w jego posiadaniu. W rozwiązaniu tym nie występuje problem blokad i głodzenia procesów, gdyż żeton jest przesyłany między procesami cyklicznie w kolejności określonej przez strukturę pierścienia.